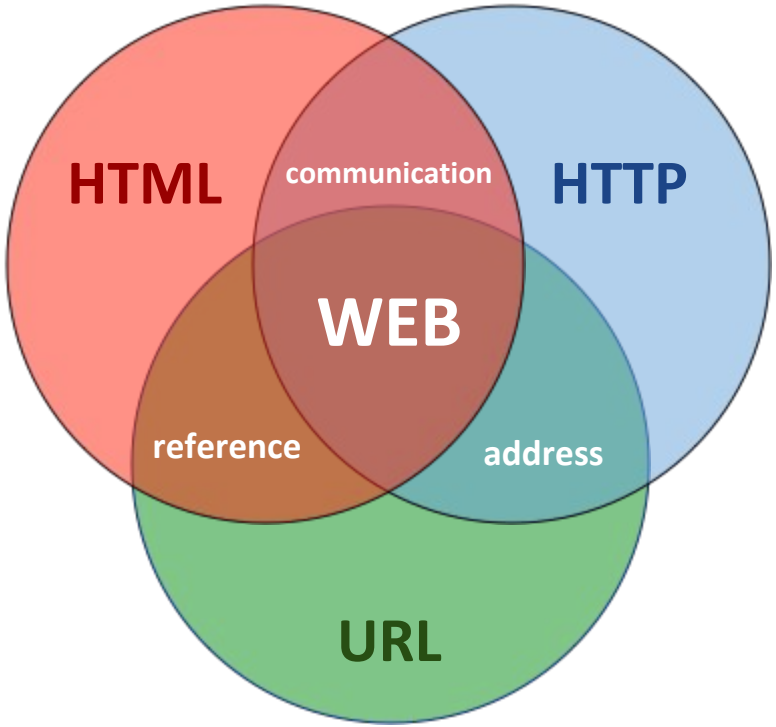
The Architecture of the Web



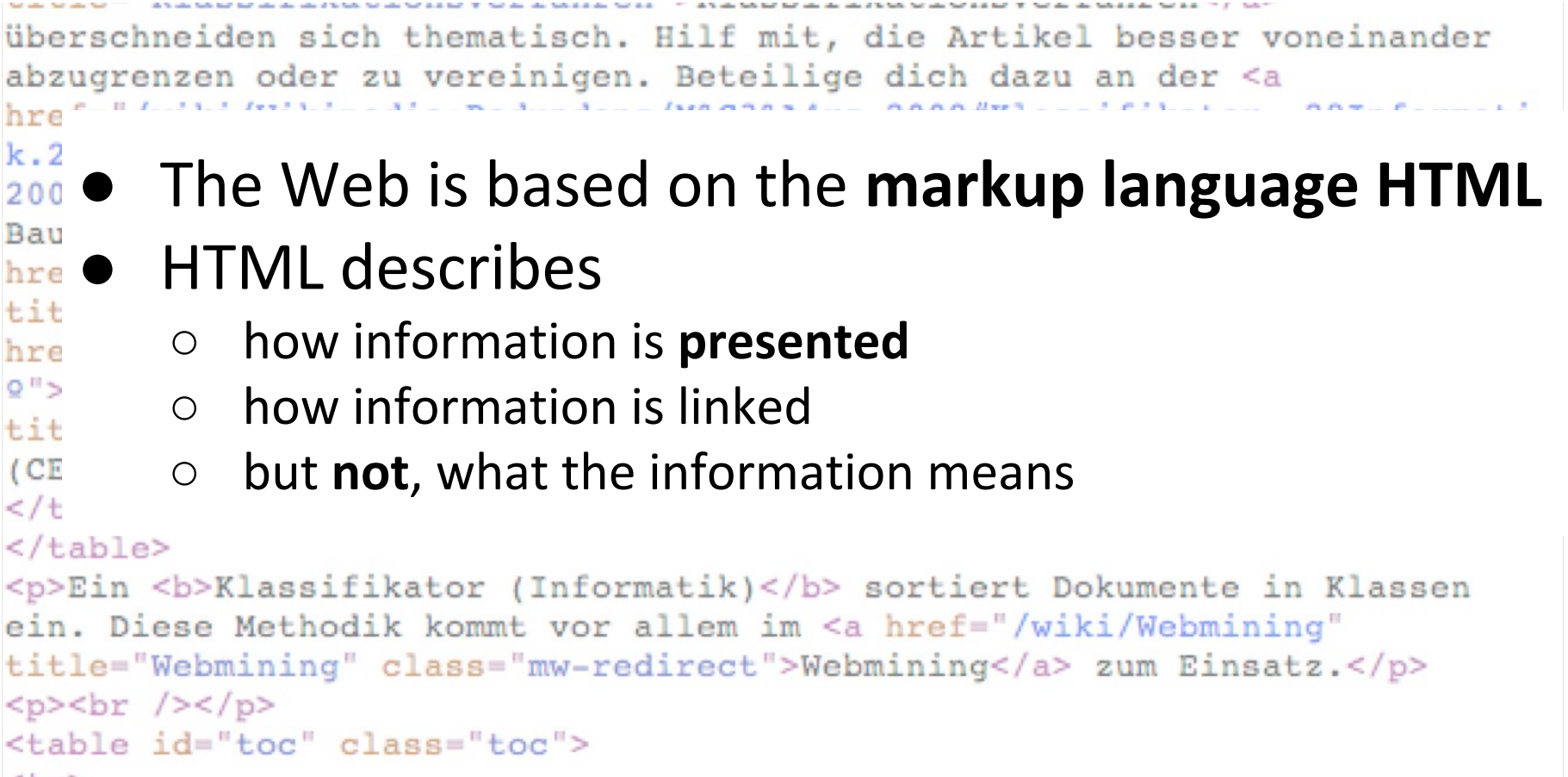
1. Identification (URI) & address (**URL**)
   e.g. `http://hpi.de`

2. Communication / protocol (**HTTP**)
   ```
   GET /index HTTP/2
   Host: hpi.de
   ```

3. Representation language (**HTML**)
   ```
   Tabea works at
   <a href="http://hpi.de">HPI</a>.
   ```

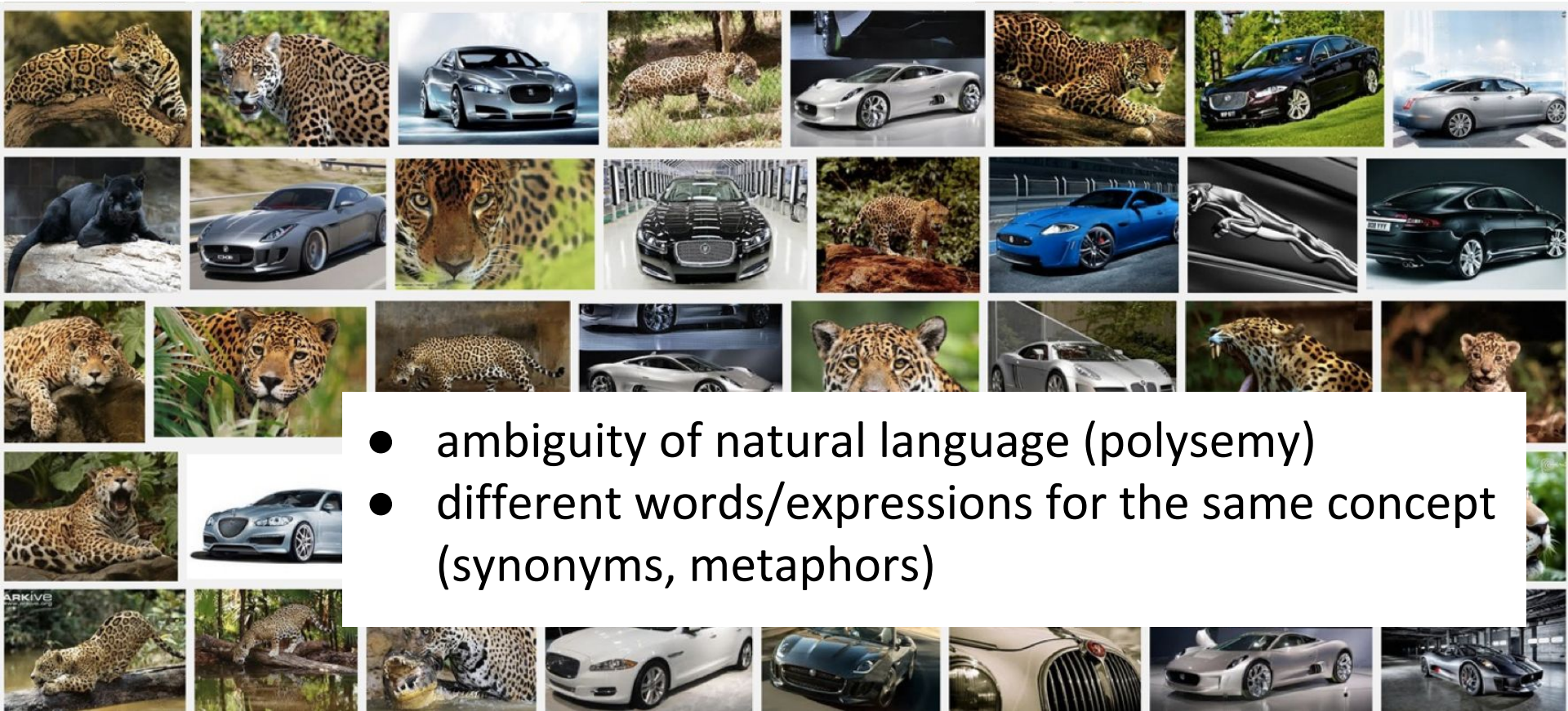
Thanks to http://www.slideshare.net/fabien_gandon

The (Document) Web is for Humans



- The Web is based on the **markup language HTML**
- HTML describes
  - how information is **presented**
  - how information is linked
  - but **not**, what the information means

The Information Retrieval Dilemma

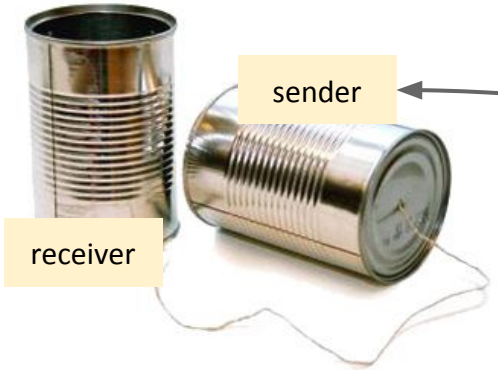


- ambiguity of natural language (polysemy)
- different words/expressions for the same concept (synonyms, metaphors)

Meaning and Comprehension

- **Understanding** is the ability to grasp the **meaning of information**.
- Information is conveyed in a **message** using a **specific language**.
- Information is understood by the receiver of a message, if the receiver **interprets** the information **correctly**.

Communication of Meaning



sender

receiver

Experience

Concept

symbolizes

refers to

Context

Experience

Symbol

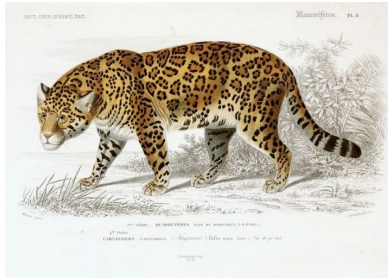
Object

stands for

"Jaguar"

Pragmatics







Ogden, Richards: The **Meaning of Meaning**: A Study of the Influence of Language upon Thought and of the Science of Symbolism, 1923

Understanding Data on the Web



https://solarsystem.nasa.gov/multimedia/gallery/01_Stern_03_Pluto_Color_TXT.jpg

Understanding Data on the Web



Text: ”**Pluto**“

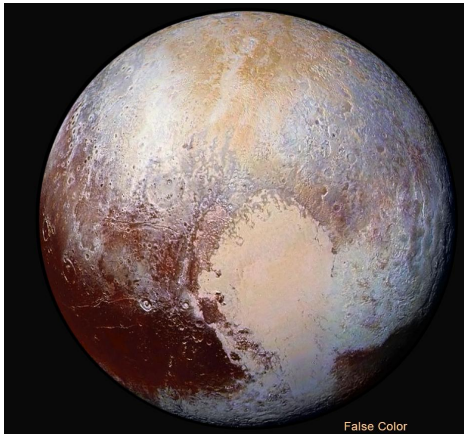
Entity Mapping
Disambiguation

Pluto	a Disney cartoon character
Pluto	a Roman god
Pluto	a song by Björk
HMS Pluto	a ship
...	
Pluto	a dwarf planet

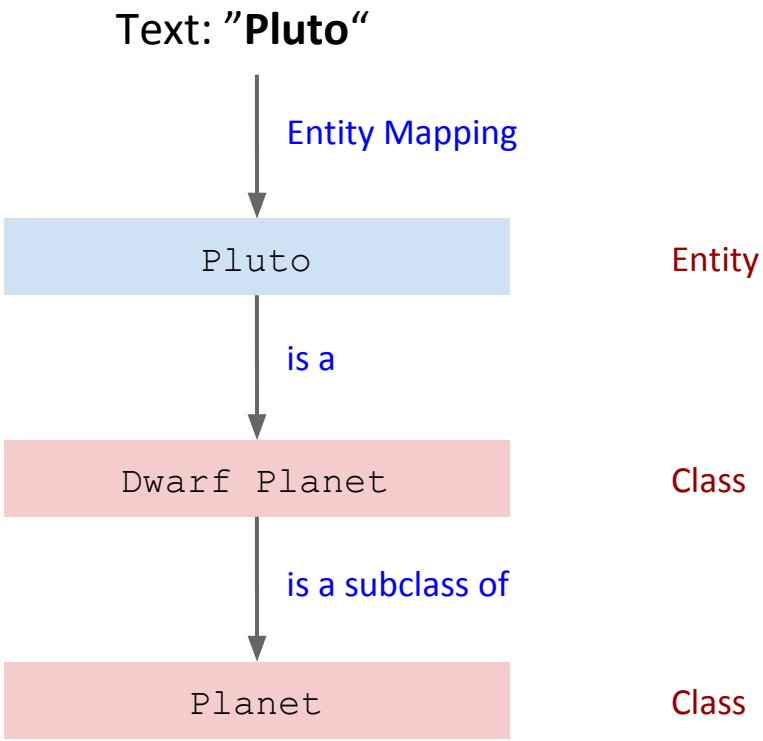
Disambiguation

- solution of linguistic ambiguities

Understanding Content on the Web

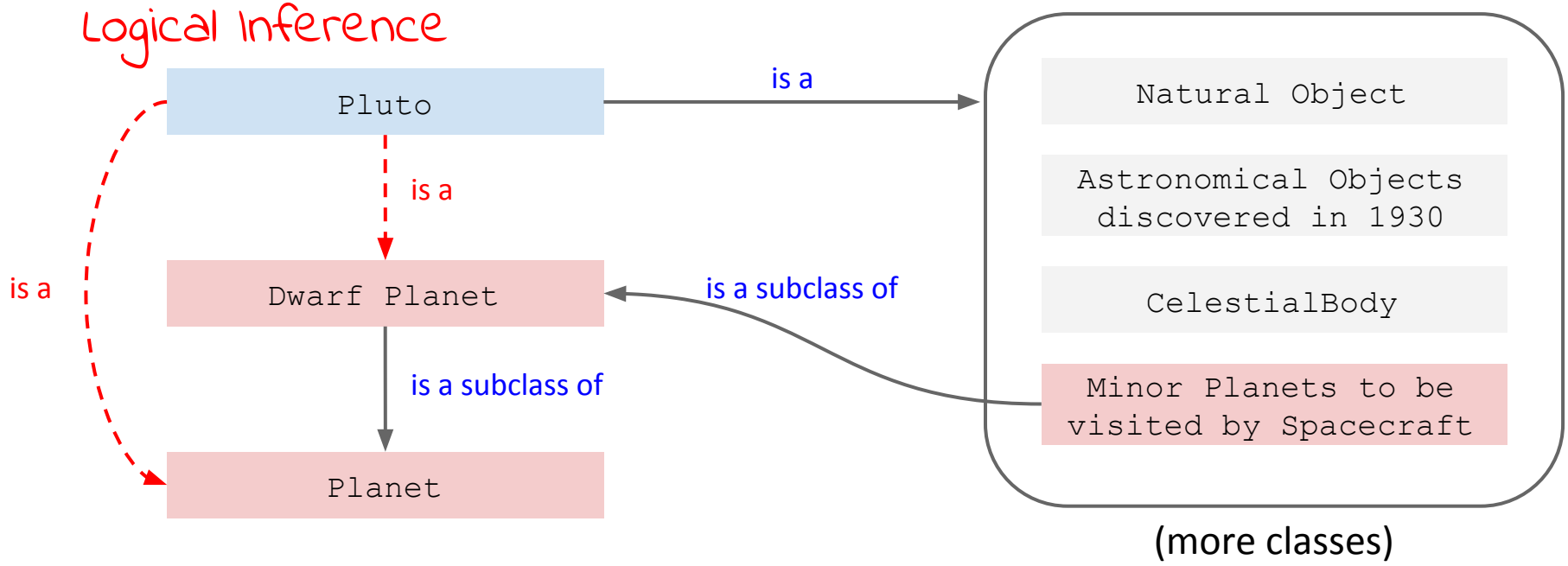


- The **Meaning (Semantics)** of entities and classes must be defined explicitly.



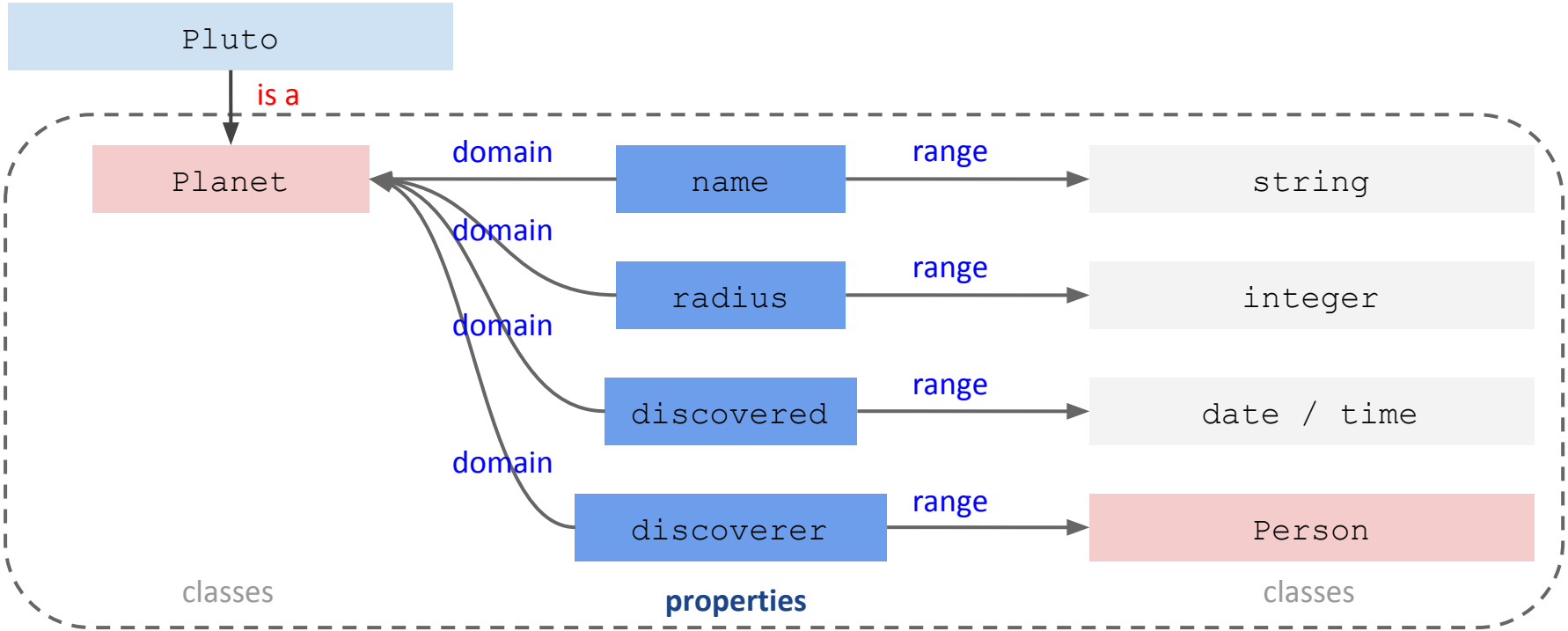
Understanding Content on the Web

- The Meaning (Semantics) is expressed with the help of knowledge representations (**Ontologies**)



Understanding Content on the Web

- The Meaning (Semantics) is expressed with the help of knowledge representations (**Ontologies**)

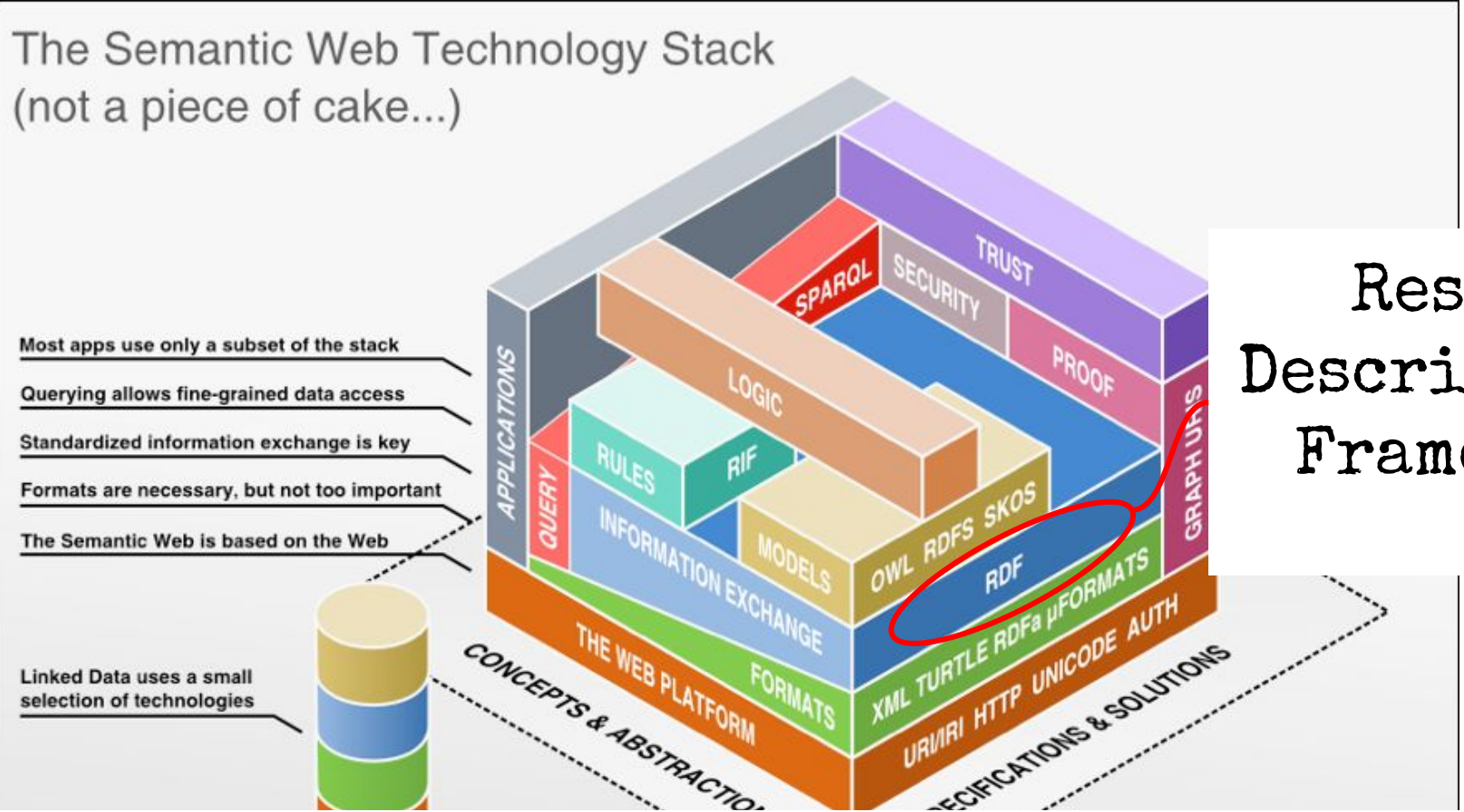


The Semantic Web - A Web of Data

- The meaning of information (Semantics) is made explicit by **formal (structured) and standardized knowledge representations (Ontologies)**.

- Thus it will be possible,
  - to process the meaning of information automatically
  - to relate and integrate heterogeneous data
  - to deduce implicit (not evident) information from existing (evident) information in an automated way

- The Semantic Web is kind of a **global database** that contains a **universal network of semantic propositions**.



Resource Description Framework (RDF)

Next: 03 - Towards a Universal Data Representation

Knowledge Representation

...a simple example

- How do I represent the following fact:
  "*Pluto has been discovered in 1930*"?



Pluto : Planet
discovered = 1930

UML instance

```
<a href="http://en.wikipedia.org/wiki/Pluto">
 Pluto
</a> has been discovered in 1930.
```

HTML

```
<planet name = "Pluto" discovered="1930" />
```

XML

Knowledge Representation

...a simple example

- How do I represent the following fact:
  "*Pluto has been discovered in 1930*" in **XML**?



```
<discovered>
  <discovery>Pluto</discovery>
  <year>1930</year>
</discovered>
```

```
<planet name = "Pluto">
  <yearDiscovered>1930</yearDiscovered>
</planet>
```

```
<planet name = "Pluto" discovered="1930" />
```

is there a unique (intuitive) way to model knowledge (in XML)?

Knowledge Representation

...with XML?

- In XML successful communication of information requires translation among different XML serializations

```
<discovered>
  <discovery>Pluto</discovery>
  <year>1930</year>
</discovered>
```

XML Schema A

XSLT Transformation

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://....Transform" >
 <xsl:template match="/">
 ....
 </xsl:template>
</xsl:stylesheet>
```

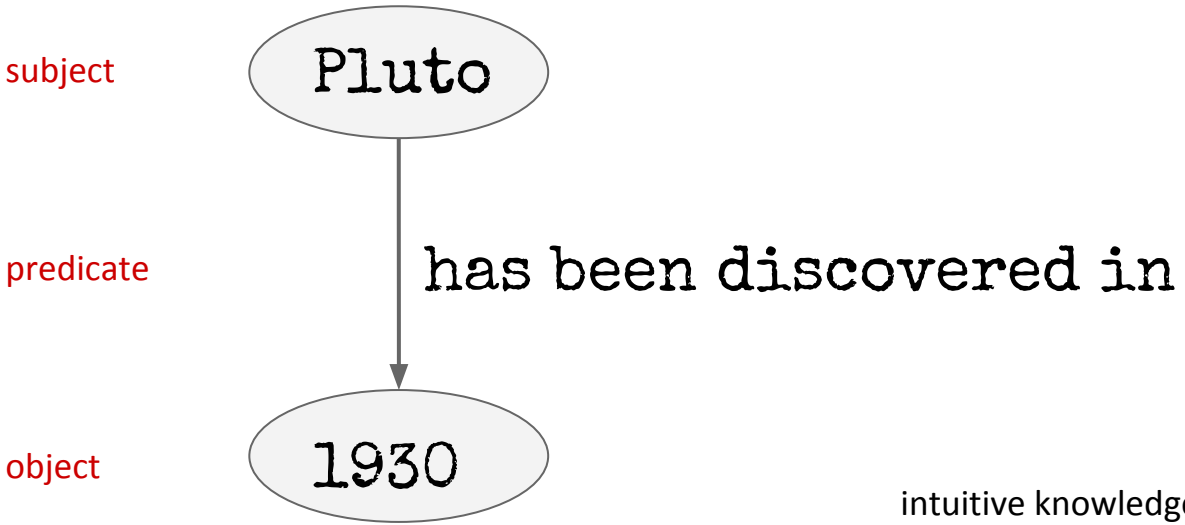
XML Schema B

```
<planet name = "Pluto" discovered="1930" />
```

Knowledge Representation

...a simple example

- How do I represent the following fact:
  "*Pluto has been discovered in 1930*" in an intuitive way?



intuitive knowledge representation with a **directed graph**



Resource Description Framework

- **Resource**
  - can be everything
  - must be uniquely **identified** and **referenceable** via **URI**
- **Description**
  - = description of resources
  - via representing properties and relationships among resources as **graphs**
- **Framework**
  - = combination of web based protocols (URI, HTTP, XML, Turtle, JSON, …)
  - based on formal model (semantics)

- Knowledge in RDF is expressed as a **list of statements**
- all RDF statements follow the same simple schema (= **RDF Triple**)

Resource Description Framework



Resource Description Framework

- **RDF Statements (RDF-Triple):**

Subject	+	Property	+	Object / Value
URI		**URI**		**URI / Literal**

RDF Building Blocks

N-Triples Serialization

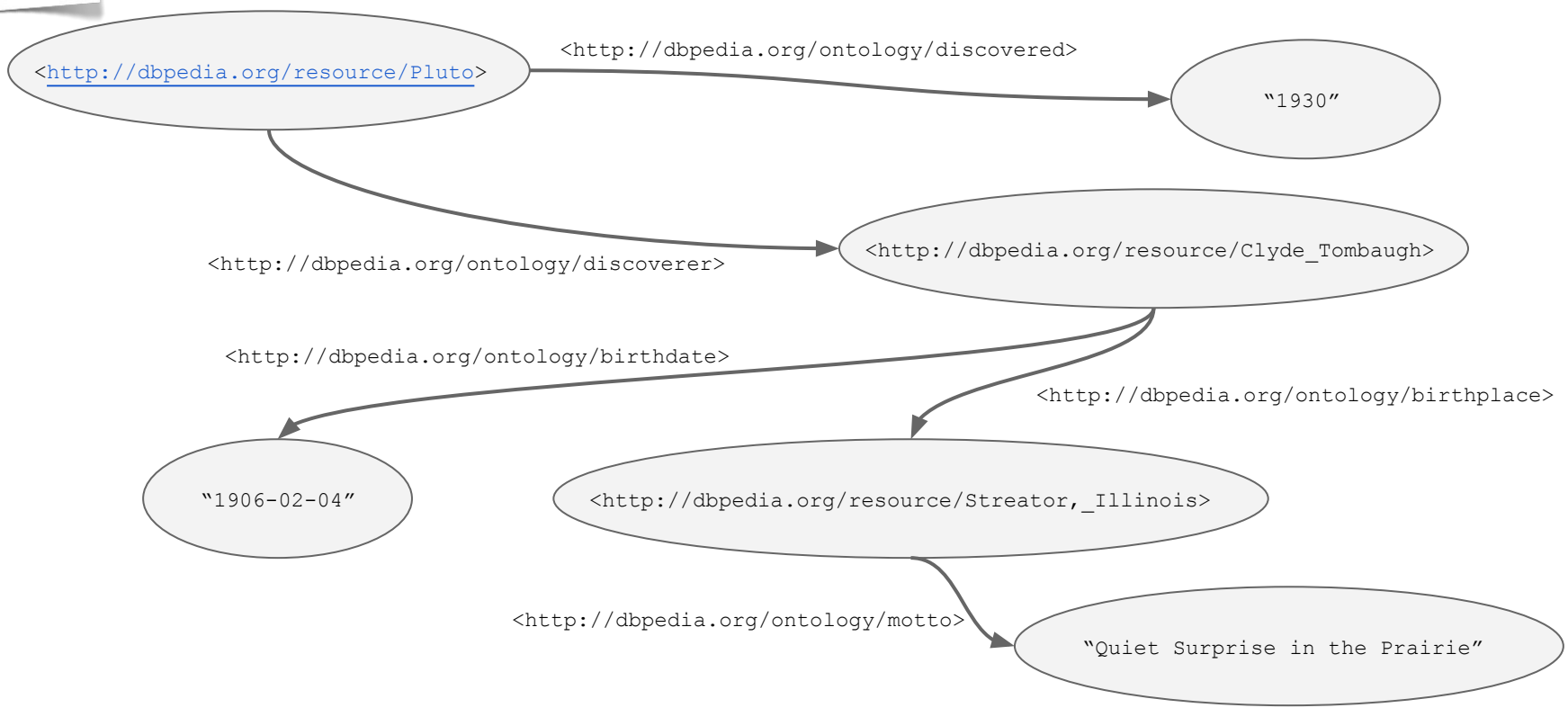
```
<http://dbpedia.org/resource/Pluto>  <http://dbpedia.org/ontology/discovered>  "1930" .
```



graph representation

Resource Description Framework

From Triple to Knowledge Graph





Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> “1930” .
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer> <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/CelestialBody> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/place> .
… … …

<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthdate> “1906-02-04” .
<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthplace> <http://dbpedia.org/resource/Streator,_Illinois> .
… … …

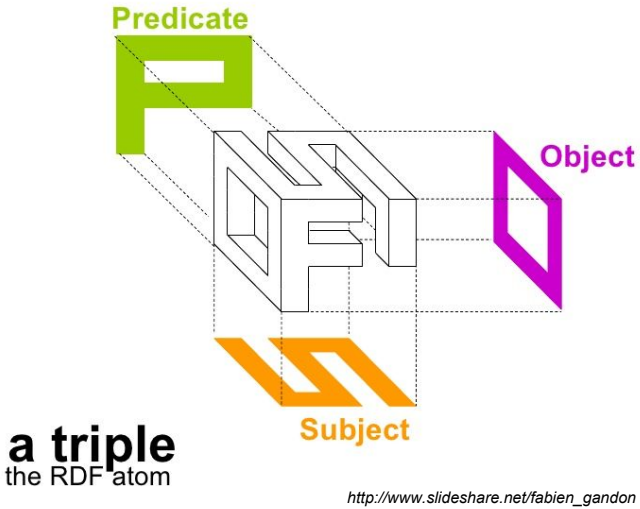
<http://dbpedia.org/resource/Streator,_Illinois> <http://dbpedia.org/ontology/motto> “Quiet Surprise in the Prairie” .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> “41.120834”^^xsd:float .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#long> “-88.835281”^^xsd:float .
… … …

Subject **Property** **Object**

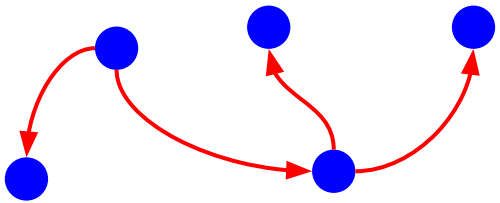
RDF Triples

Resource Description Framework

RDF is a **Triple Model**
i.e. every piece of knowledge is broken down into
(**subject** , **predicate** , **object**)



RDF is a **Graph Model**
i.e. every piece of knowledge is interpreted as
(**vertex** , **edge** , **vertex**)





Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> “1930” .
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer> <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/CelestialBody> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/place> .
… … …

<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthdate> “1906-02-04” .
<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthplace> <http://dbpedia.org/resource/Streator,_Illinois> .
… … …

<http://dbpedia.org/resource/Streator,_Illinois> <http://dbpedia.org/ontology/motto> “Quiet Surprise in the Prairie” .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> “41.120834”^^xsd:float .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#long> “-88.835281”^^xsd:float .
… … …

Individuals (Entities)



Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> “1930” .
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer> <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> **<http://dbpedia.org/ontology/CelestialBody>** .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> **<http://schema.org/place>** .
… … …

<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthdate> “1906-02-04” .
<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthplace> <http://dbpedia.org/resource/Streator,_Illinois> .
… … …

<http://dbpedia.org/resource/Streator,_Illinois> <http://dbpedia.org/ontology/motto> “Quiet Surprise in the Prairie” .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> “41.120834”^^xsd:float .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#long> “-88.835281”^^xsd:float .
… … …

Classes



Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> **“1930”** .
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer> <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/CelestialBody> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/place> .
… … …

<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthdate> **“1906-02-04”** .
<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthplace> <http://dbpedia.org/resource/Streator,_Illinois> .
… … …

<http://dbpedia.org/resource/Streator,_Illinois> <http://dbpedia.org/ontology/motto> “Quiet Surprise in the Prairie” .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> **“41.120834”^^xsd:float** .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#long> **“-88.835281”^^xsd:float** .
… … …

Literals



Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> **<http://dbpedia.org/ontology/discovered>** "1930" .
<http://dbpedia.org/resource/Pluto> **<http://dbpedia.org/ontology/discoverer>** <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> **<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>** <http://dbpedia.org/ontology/CelestialBody> .
<http://dbpedia.org/resource/Pluto> **<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>** <http://schema.org/place> .
... ...

<http://dbpedia.org/resource/Clyde_Tombaugh> **<http://dbpedia.org/ontology/birthdate>** "1906-02-04" .
<http://dbpedia.org/resource/Clyde_Tombaugh> **<http://dbpedia.org/ontology/birthplace>** <http://dbpedia.org/resource/Streator,_Illinois> .
... ...

<http://dbpedia.org/resource/Streator,_Illinois> **<http://dbpedia.org/ontology/motto>** "Quiet Surprise in the Prairie" .
<http://dbpedia.org/resource/Streator,_Illinois> **<http://www.w3.org/2003/01/geo/wgs84_pos#lat>** "41.120834"^^xsd:float .
<http://dbpedia.org/resource/Streator,_Illinois> **<http://www.w3.org/2003/01/geo/wgs84_pos#long>** "-88.835281"^^xsd:float .
... ...

Properties



Resource Description Framework

From Triple to Knowledge Graph

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> “1930” .
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer> <http://dbpedia.org/resource/Clyde_Tombaugh> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/CelestialBody> .
<http://dbpedia.org/resource/Pluto> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/place> .
… … …

<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthdate> “1906-02-04” .
<http://dbpedia.org/resource/Clyde_Tombaugh> <http://dbpedia.org/ontology/birthplace> <http://dbpedia.org/resource/Streator,_Illinois> .
… … …

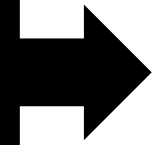
<http://dbpedia.org/resource/Streator,_Illinois> <http://dbpedia.org/ontology/motto> “Quiet Surprise in the Prairie” .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#lat> “41.120834”^^xsd:float .
<http://dbpedia.org/resource/Streator,_Illinois> <http://www.w3.org/2003/01/geo/wgs84_pos#long> “-88.835281”^^xsd:float .
… … …

Vocabularies / Ontologies

URL

identify
what exists
on the web

http://www.mywebsite.org/



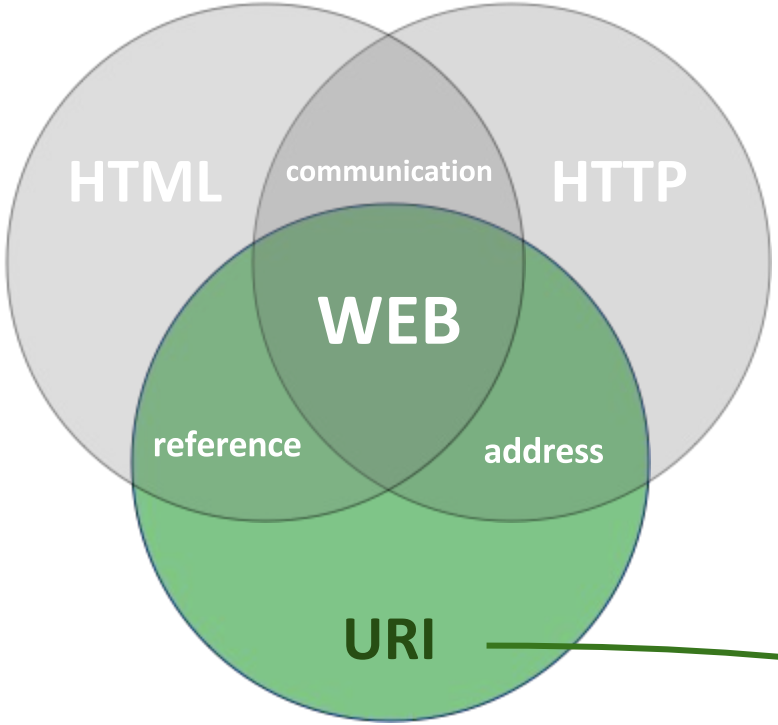
URI

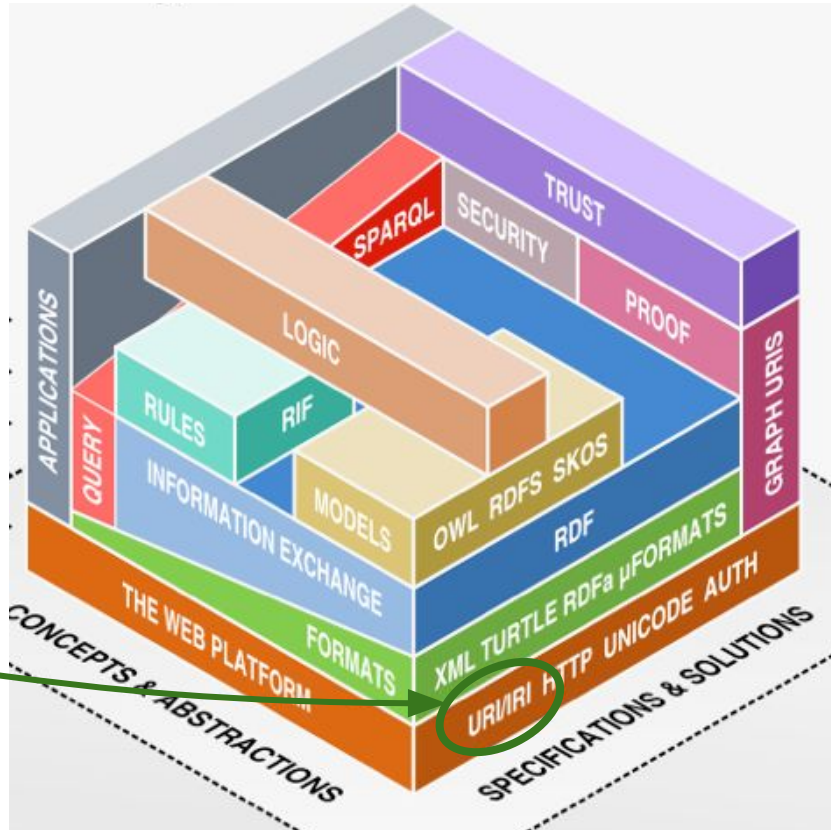
identify
on the web
what exists

http://hpi.de/harald_foaf.rdf#me

Thanks to http://www.slideshare.net/fabien_gandon

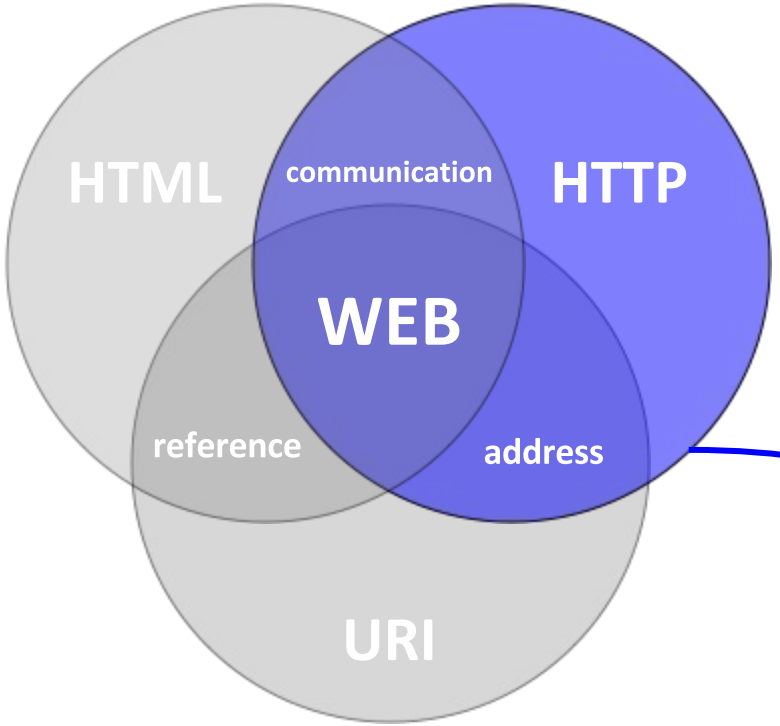
Linked Data and the Web

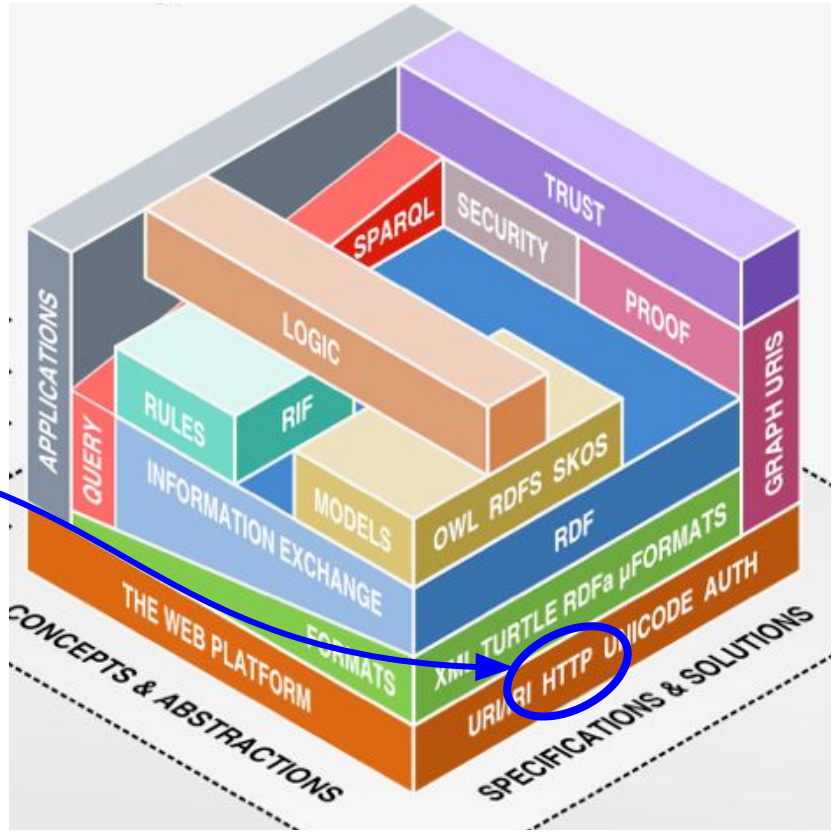




Thanks to http://www.slideshare.net/fabien_gandon

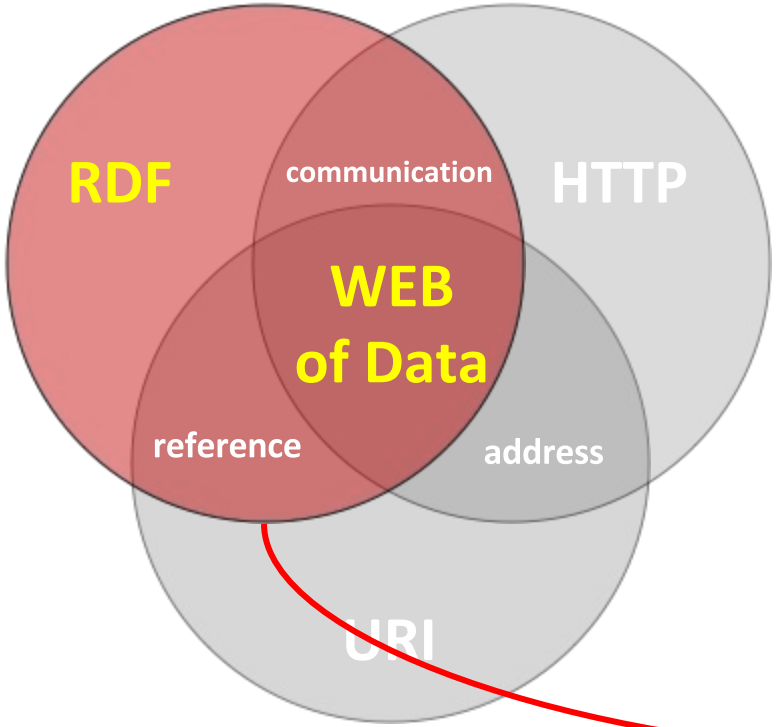
Linked Data and the Web

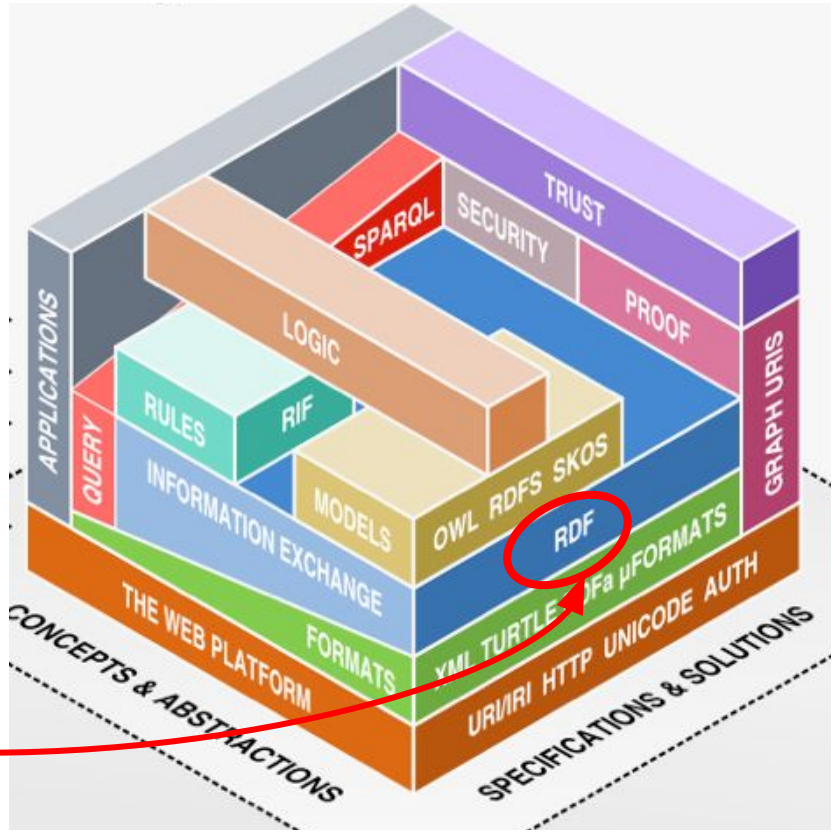




Thanks to http://www.slideshare.net/fabien_gandon

Linked Data and the Web





Thanks to http://www.slideshare.net/fabien_gandon

Resource Description Framework

- **RDF Statements (RDF Triple):**

Subject	+	Property	+	Object / Value
URI		**URI**		**URI / Literal**

RDF Building Blocks

N-Triples Serialization

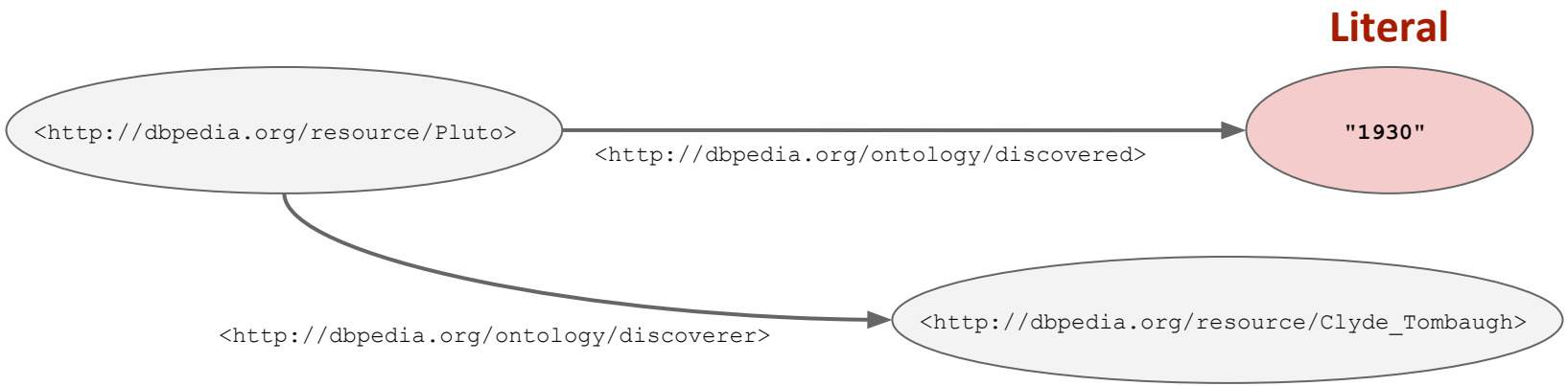
```
<http://dbpedia.org/resource/Pluto>    <http://dbpedia.org/ontology/discovered>    "1930" .
```



graph representation

Resource Description Framework

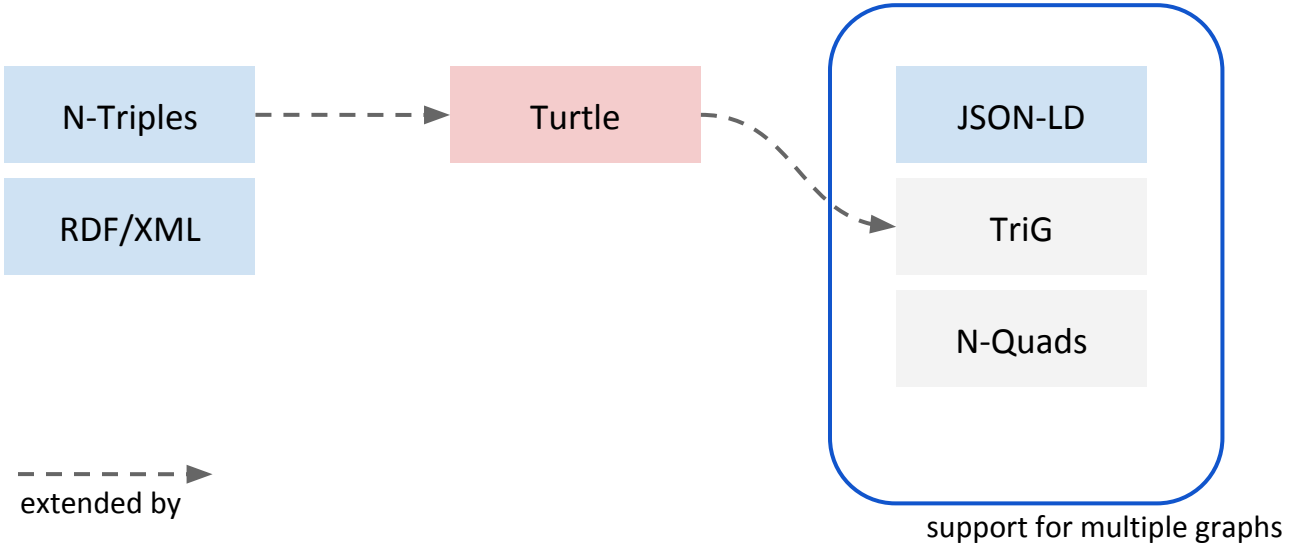
- **URIs and Literals**
  - **URIs** reference resources uniquely
  - **Literals** describe data values that don't have a separate existence



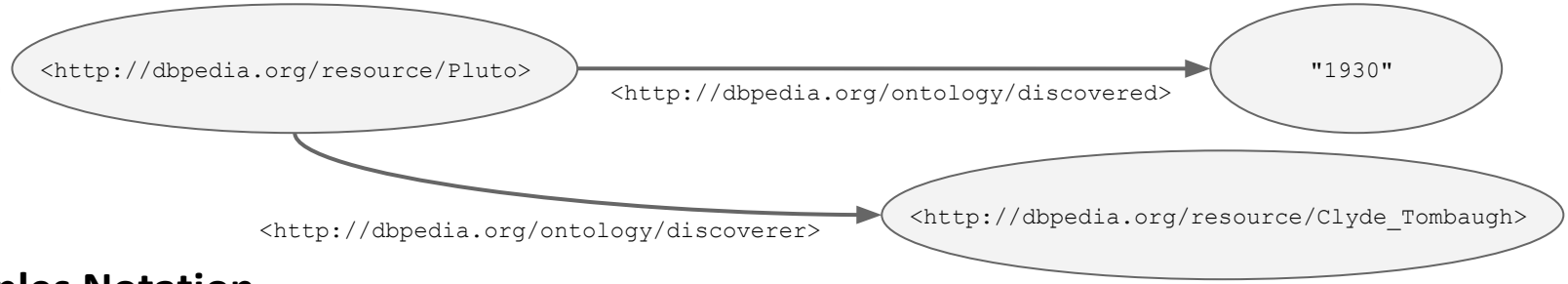


RDF Serializations

- RDF comes with several different **serialization formats**:
  - N-Triples, RDF/XML, JSON, Turtle, TriG, N-Quads, RDFa, ...



RDF Serializations

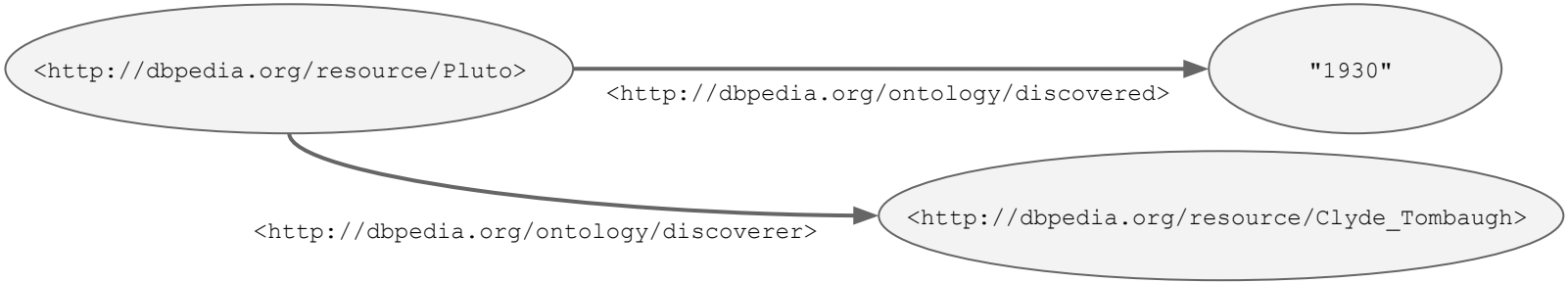


- **N-Triples Notation**
  - **URIs/IRIs** in angle brackets
  - **Literals** in quotation marks
  - Triple ends with a **period**

```
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> "1930" .

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer>
<http://dbpedia.org/resource/Clyde_Tombaugh> .
```

RDF Serializations

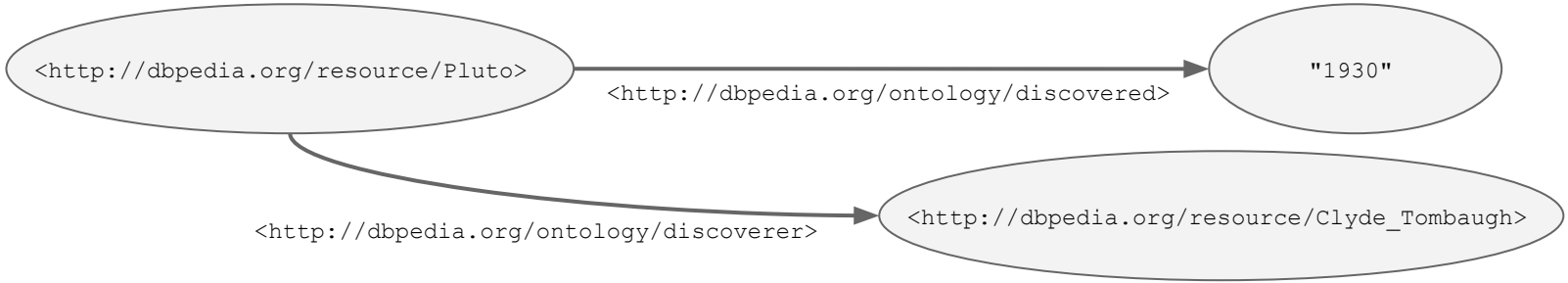


- **RDF/XML Notation**

S P O

```
<?xml version="1.0" encoding="utf-8" ?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ns0="http://dbpedia.org/ontology/">
  <rdf:Description rdf:about="http://dbpedia.org/resource/Pluto">
    <ns0:discovered>1930</ns0:discovered>
    <ns0:discoverer rdf:resource="http://dbpedia.org/resource/Clyde_Tombaugh"/>
  </rdf:Description>
</rdf:RDF>
```

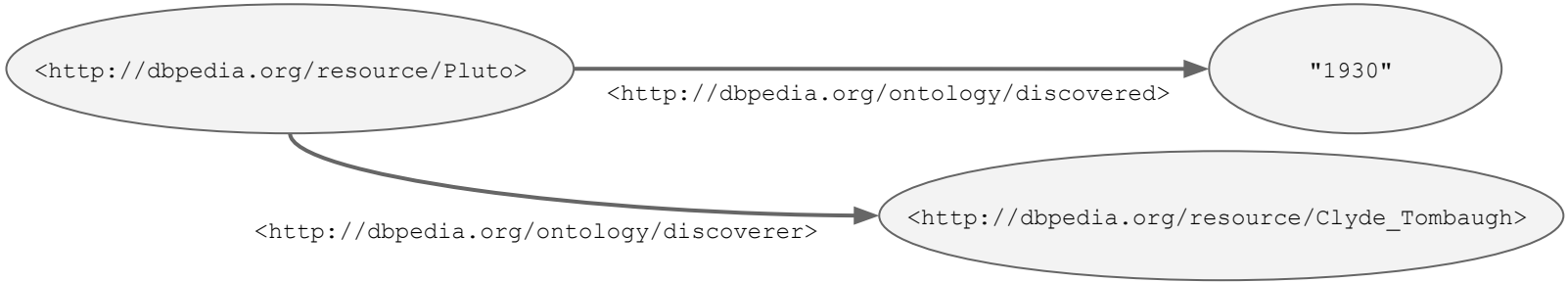
RDF Serializations



- **JSON-LD Notation (RDF 1.1)** S P O

```
{    "@id" :    "http://dbpedia.org/resource/Pluto" ,
     "http://dbpedia.org/ontology/discovered":
            { "@value" : "1930" }
  ,
     "http://dbpedia.org/ontology/discoverer" :
            { "@id" : "http://dbpedia.org/resource/Clyde_Tombaugh"}
}
```

RDF Serializations



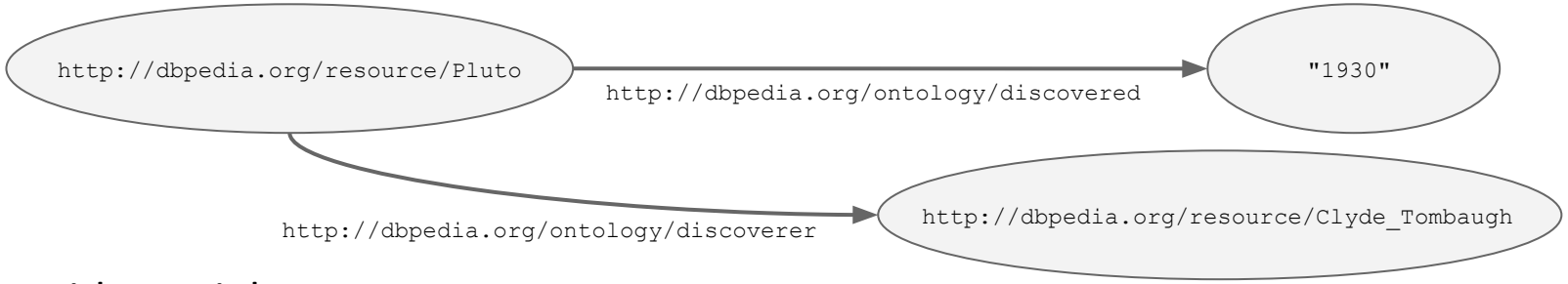
- **Turtle (Terse RDF Triple Language) Notation**
  - extension of N-Triples

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@base  <http://dbpedia.org/resource/> .

<Pluto> dbo:discovered "1930" .

<Pluto> dbo:discoverer <Clyde_Tombaugh> .
```

Turtle - Terse RDF Triple Language



- starting with N-Triples

```
<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discovered> "1930" .

<http://dbpedia.org/resource/Pluto> <http://dbpedia.org/ontology/discoverer>
<http://dbpedia.org/resource/Clyde_Tombaugh> .
```

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@base  <http://dbpedia.org/resource/> .

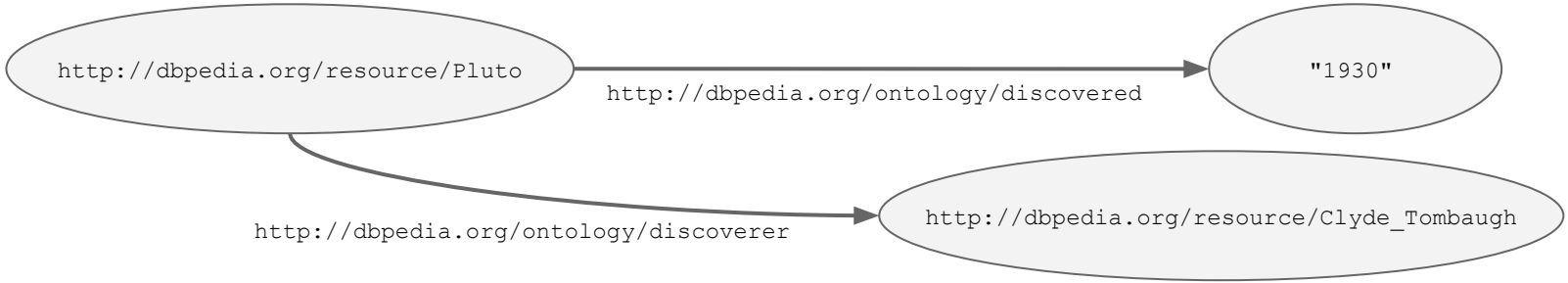
<Pluto> dbo:discovered "1930" .

<Pluto> dbo:discoverer <Clyde_Tombaugh> .
```

@prefix directive associates prefix-label with URI

@base directive provides URI to complement all relative URIs

Turtle - Terse RDF Triple Language



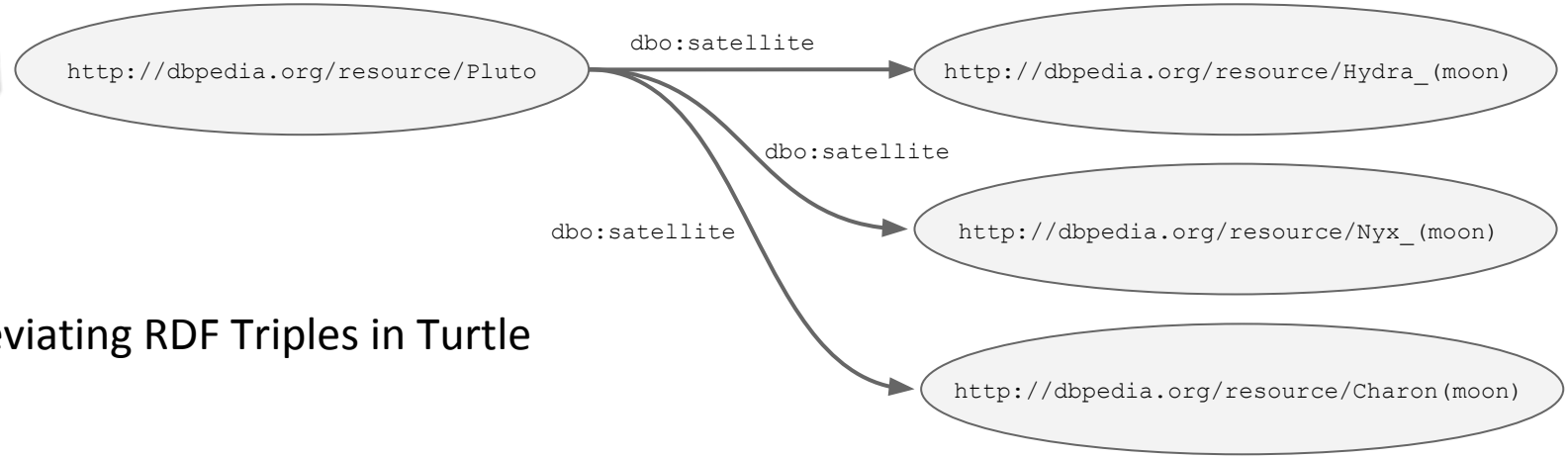
- Abbreviating RDF Triples in Turtle

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@base  <http://dbpedia.org/resource/> .

<Pluto> dbo:discovered "1930" ;
        dbo:discoverer <Clyde_Tombaugh> .
```

semicolon indicates that subsequent triples have the same subject **(predicate list)**

Turtle - Terse RDF Triple Language



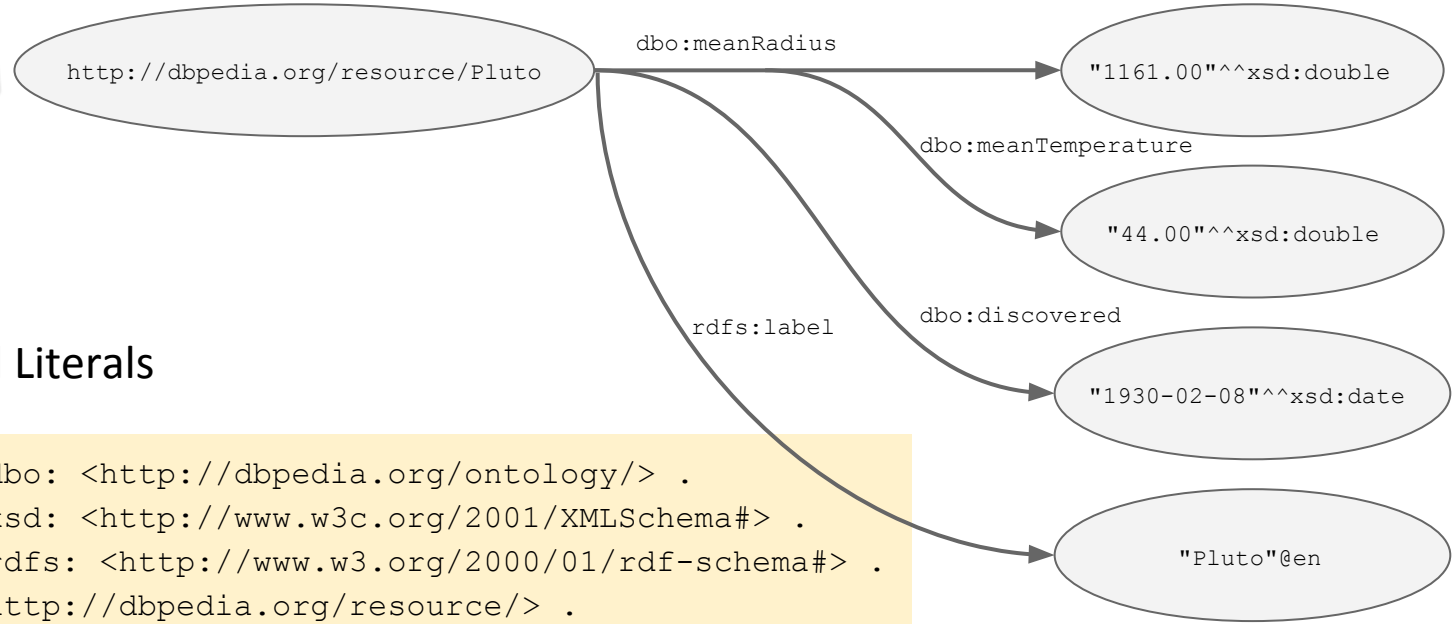
- Abbreviating RDF Triples in Turtle

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@base  <http://dbpedia.org/resource/> .

<Pluto> dbo:satellite <Hydra_(moon)> ,
                      <Nyx_(moon)> ,
                     <Charon_(moon)> .
```

comma indicates that subsequent triples have same subject and property **(object list)**

Turtle - Terse RDF Triple Language

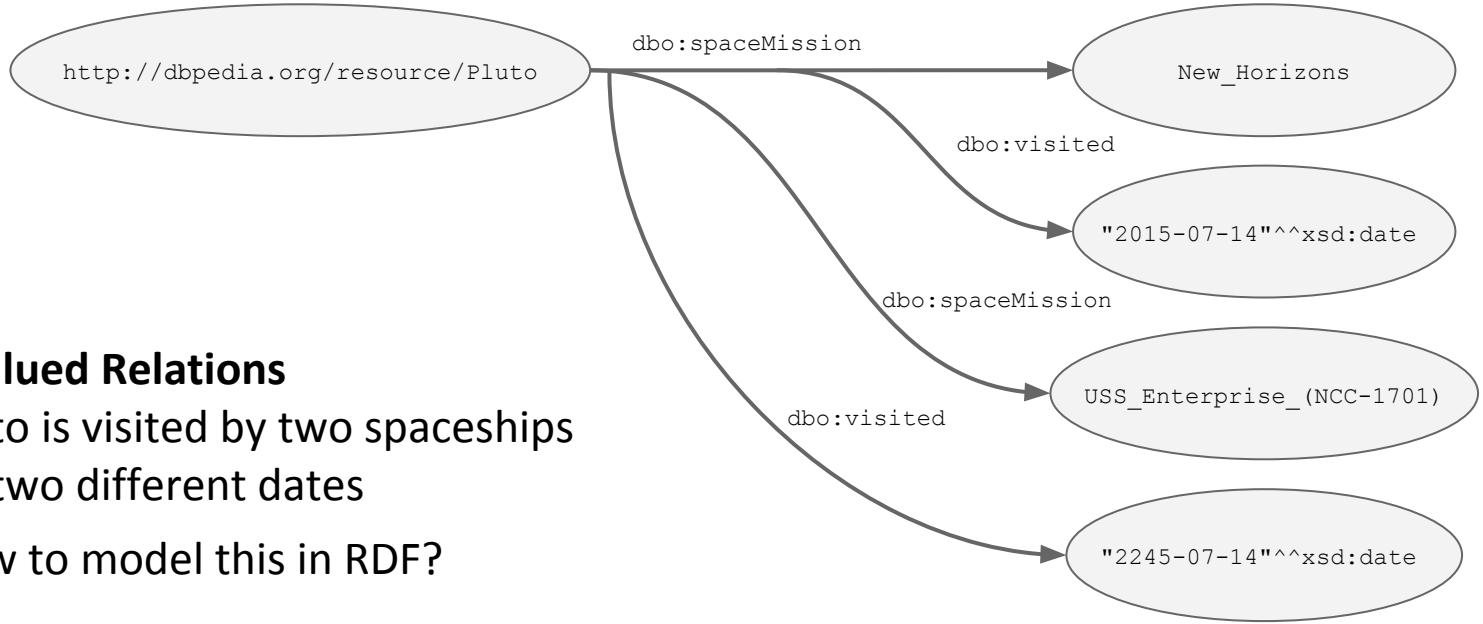


- Typed Literals

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix xsd: <http://www.w3c.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@base  <http://dbpedia.org/resource/> .

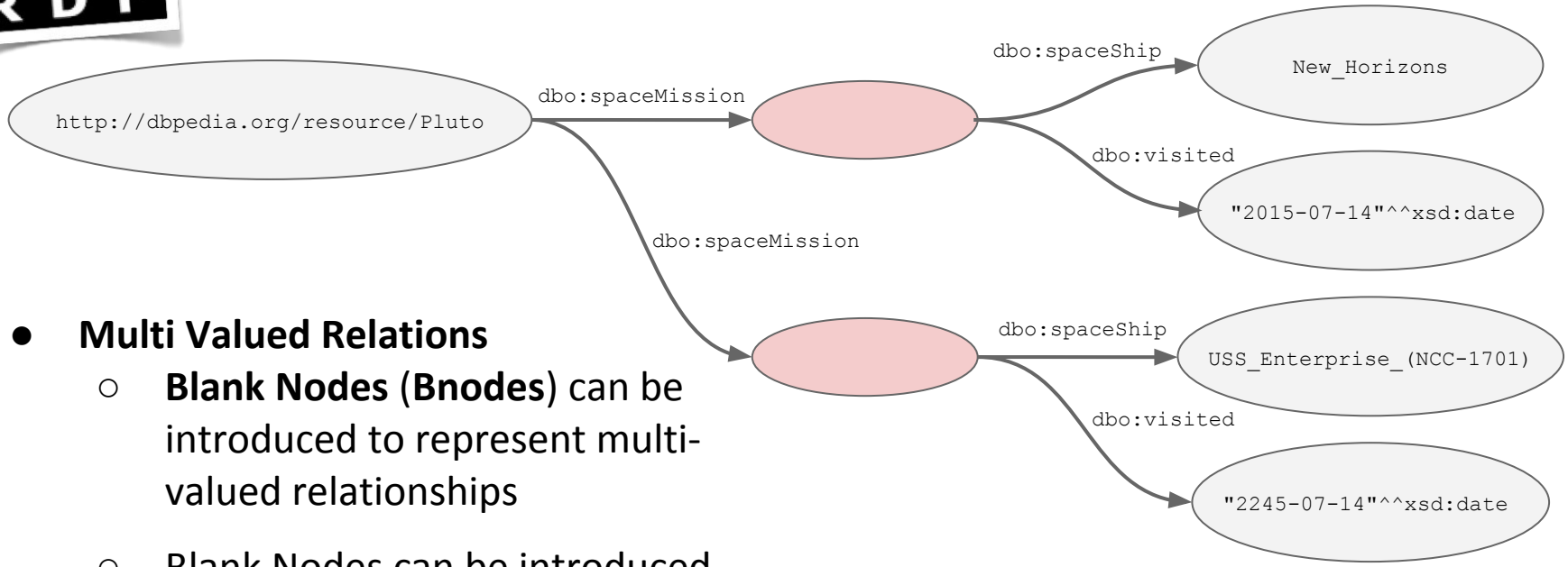
<Pluto> dbo:meanRadius "1161.00"^^xsd:double ;
        dbo:meanTemperature "44.00"^^xsd:double ;
        dbo:discovered "1930-02-18"^^xsd:date ;
        rdfs:label "Pluto"@en .
```

Turtle - Terse RDF Triple Language



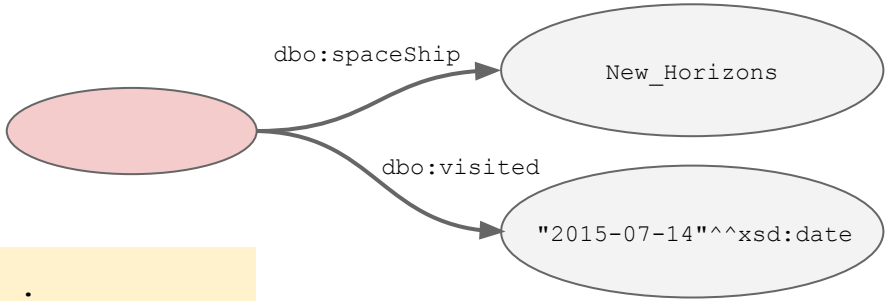
- **Multi Valued Relations**
  - Pluto is visited by two spaceships on two different dates
  - How to model this in RDF?
  - **Problem:** unique association

Turtle - Terse RDF Triple Language



- **Multi Valued Relations**
  - **Blank Nodes** (**Bnodes**) can be introduced to represent multi-valued relationships
  - Blank Nodes can be introduced for resources that don‘t need a name (auxiliary nodes)

Turtle - Terse RDF Triple Language

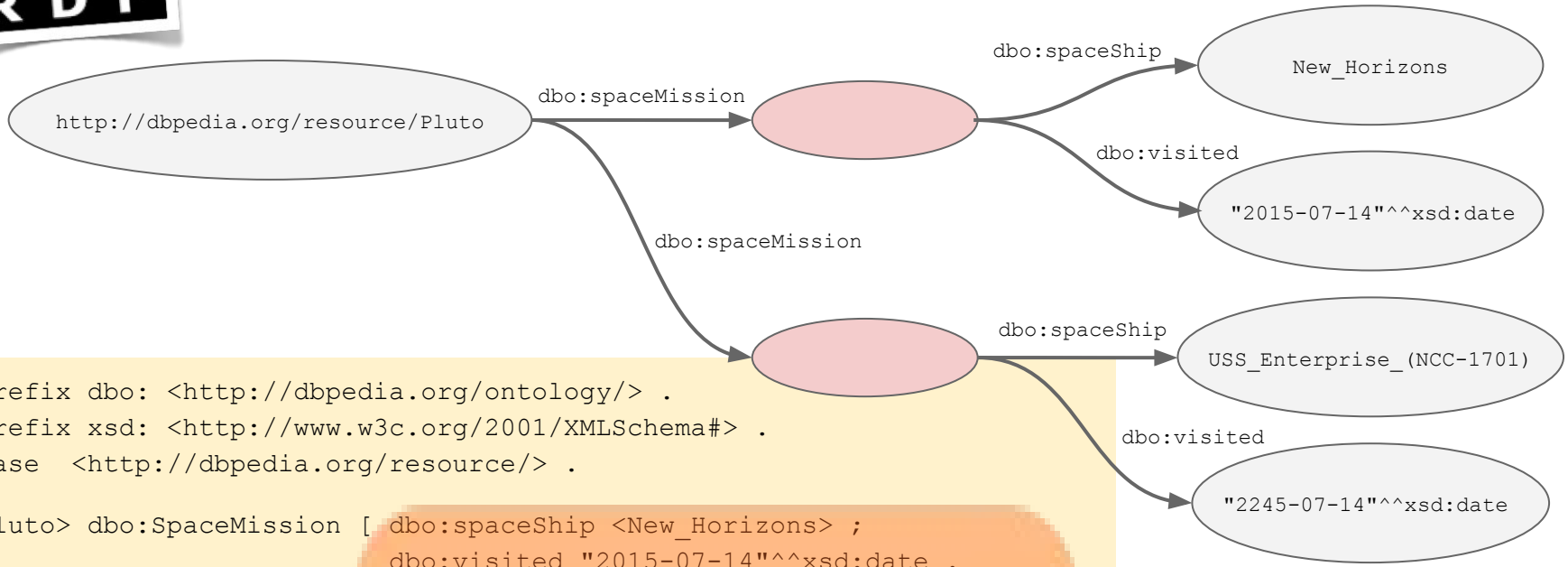



```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix xsd: <http://www.w3c.org/2001/XMLSchema#> .
@base  <http://dbpedia.org/resource/> .

[] dbo:spaceShip <New_Horizons> ;
   dbo:visited "2015-07-14"^^xsd:date .
```

anonymous blank node as subject

Turtle - Terse RDF Triple Language

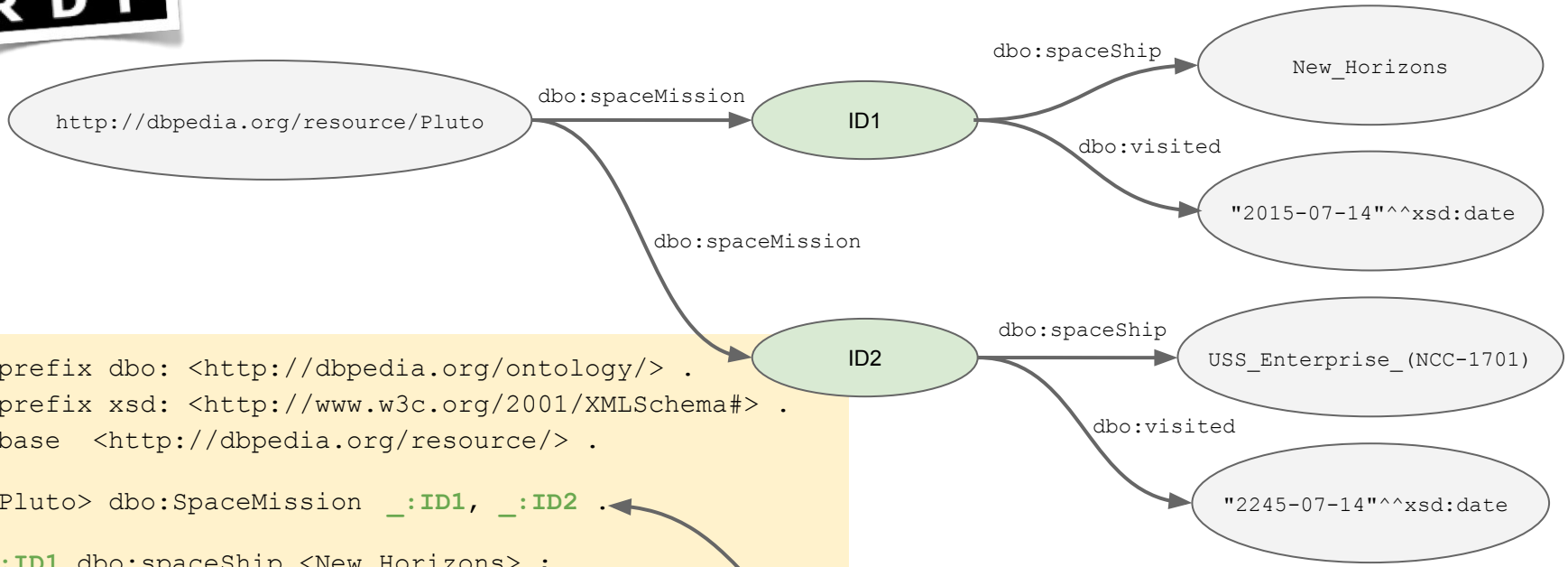



```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix xsd: <http://www.w3c.org/2001/XMLSchema#> .
@base  <http://dbpedia.org/resource/> .

<Pluto> dbo:SpaceMission [ dbo:spaceShip <New_Horizons> ;
                           dbo:visited "2015-07-14"^^xsd:date .
                         ],
                         [ dbo:spaceShip <USS_Enterprise_(NCC-1701)> ;
                           dbo:visited "2245-07-14"^^xsd:date .
                         ].
```

nested anonymous blank nodes

Turtle - Terse RDF Triple Language




```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix xsd: <http://www.w3c.org/2001/XMLSchema#> .
@base  <http://dbpedia.org/resource/> .

<Pluto> dbo:SpaceMission  _:ID1, _:ID2 .

_:ID1 dbo:spaceShip <New_Horizons> ;
      dbo:visited "2015-07-14"^^xsd:date .

_:ID2 dbo:spaceShip <USS_Enterprise_(NCC-1701)> ;
      dbo:visited "2245-07-14"^^xsd:date .
```

dereferenceable blank nodes
can only be referenced from
inside a document / graph

What does it really mean?

http://dbpedia.org/resource/Pluto

http://dbpedia.org/property/satelliteOf

http://dbpedia.org/resource/Charon_(moon)

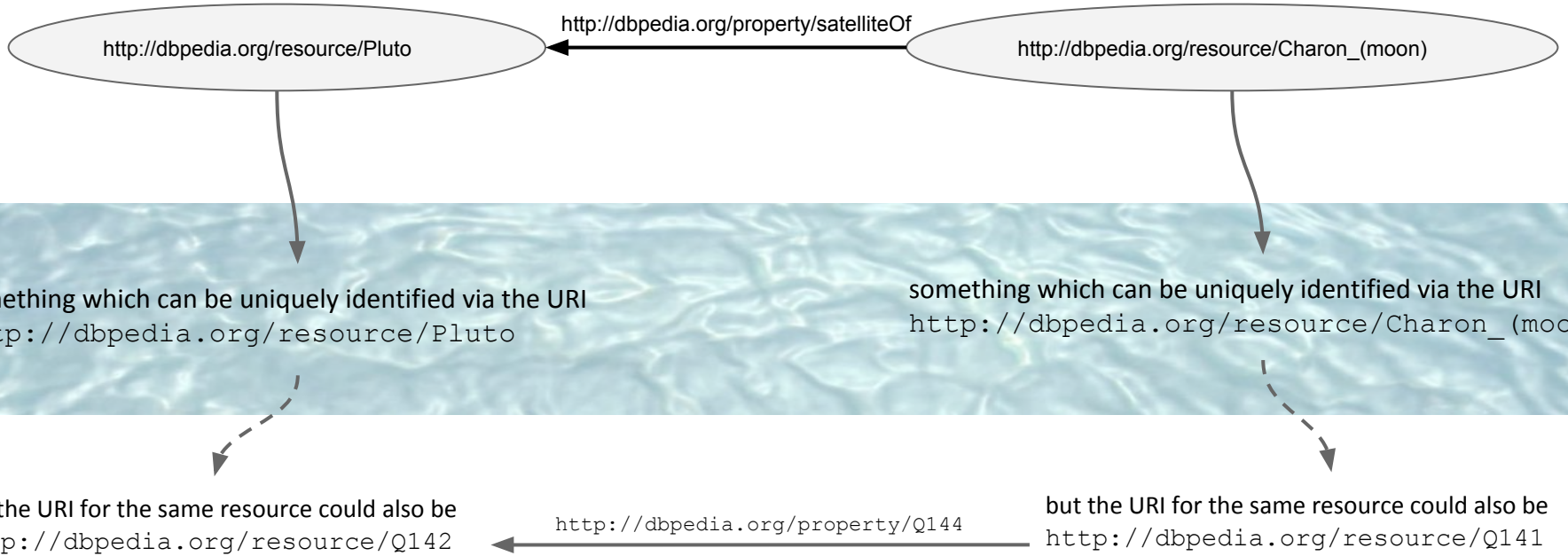
http://dbpedia.org/ontology/discoverer

http://dbpedia.org/resource/Clyde_Tombaugh



Where does the intended meaning really come from?

What does it really mean?



We need more semantic expressivity...

RDF Schema



- **RDF Schema**, officially called "**RDF Vocabulary Description Language**"

- RDF Schema allows:
  - Definition of **classes** via `rdfs:Class`
  - Class instantiation in RDF via `rdf:type`
  - Example:

```
:Planet rdf:type rdfs:Class .
:Earth  rdf:type :Planet .
```

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix :     <http://example.org/Space#> .
```

RDF Schema



- Definition of **properties** via `rdf:Property`
- Definition of **property restrictions on domain and range** via `rdfs:domain` and `rdfs:range`
- Example

```
:CelestialBody rdf:type    rdfs:Class .
:satelliteOf   rdf:type    rdf:Property .
:satelliteOf   rdfs:domain :CelestialBody .
:satelliteOf   rdfs:range  :CelestialBody .
```

RDF Schema



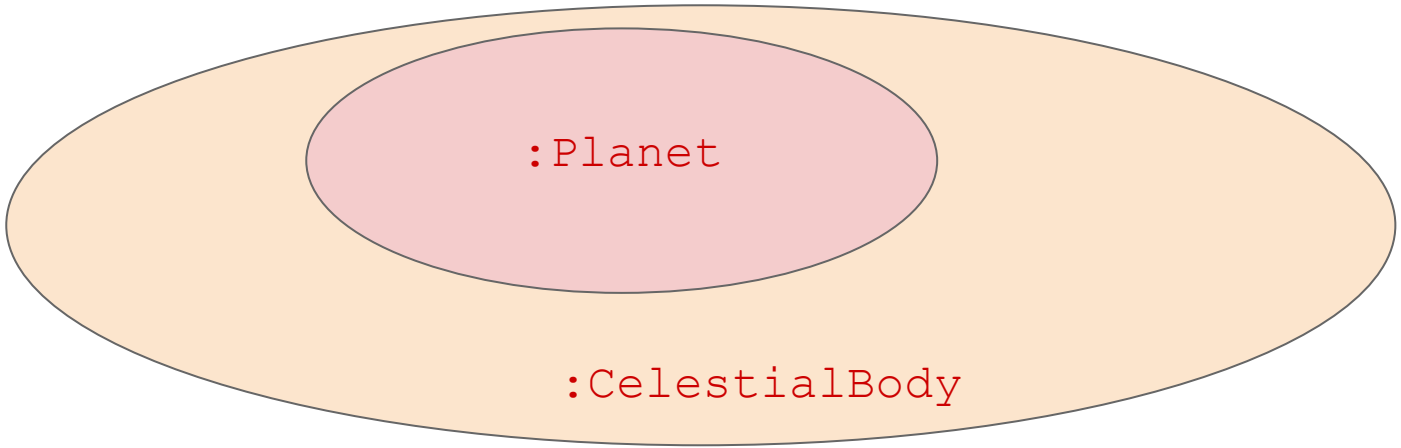
- Everything in the RDF model is a **resource**
  - `rdfs:Class rdf:type rdfs:Resource .`
  - `rdf:Property rdf:type rdfs:Resource .`
  - `rdfs:Literal rdf:type rdfs:Resource .`
  - `rdfs:XMLLiteral rdf:type rdfs:Resource .`
  - `rdfs:Datatype rdf:type rdfs:Resource .`
  - `rdfs:Container rdf:type rdfs:Resource .`
  - `rdfs:ContainerMembershipProperty rdf:type rdfs:Resource .`

RDF Schema



- Definition of **hierarchical relationships**:
  - **Subclasses** and **superclasses** via **`rdfs:subClassOf`**
  - Example:

```
:Planet rdfs:subClassOf :CelestialBody .
```



RDF Schema



- Definition of **hierarchical relationships**:
  - **Subclasses** and **superclasses** via `rdfs:subClassOf`
  - Example:

```
:Planet rdfs:subClassOf :CelestialBody .
```

  - **Subproperties** and **superproperties** via `subPropertyOf`
  - Example

```
:artificialSatelliteOf rdfs:subPropertyOf
:satelliteOf .
```

RDFS Example

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix :     <http://example.org/Space#> .
```

```
:Planet rdf:type        rdfs:Class ;
        rdfs:subClassOf :CelestialBody .
:Satellite rdf:type        rdfs:Class ;
           rdfs:subClassOf :CelestialBody .
:ArtificialSatellite rdf:type        rdfs:Class ;
                     rdfs:subClassOf :Satellite .
```

Class Definitions

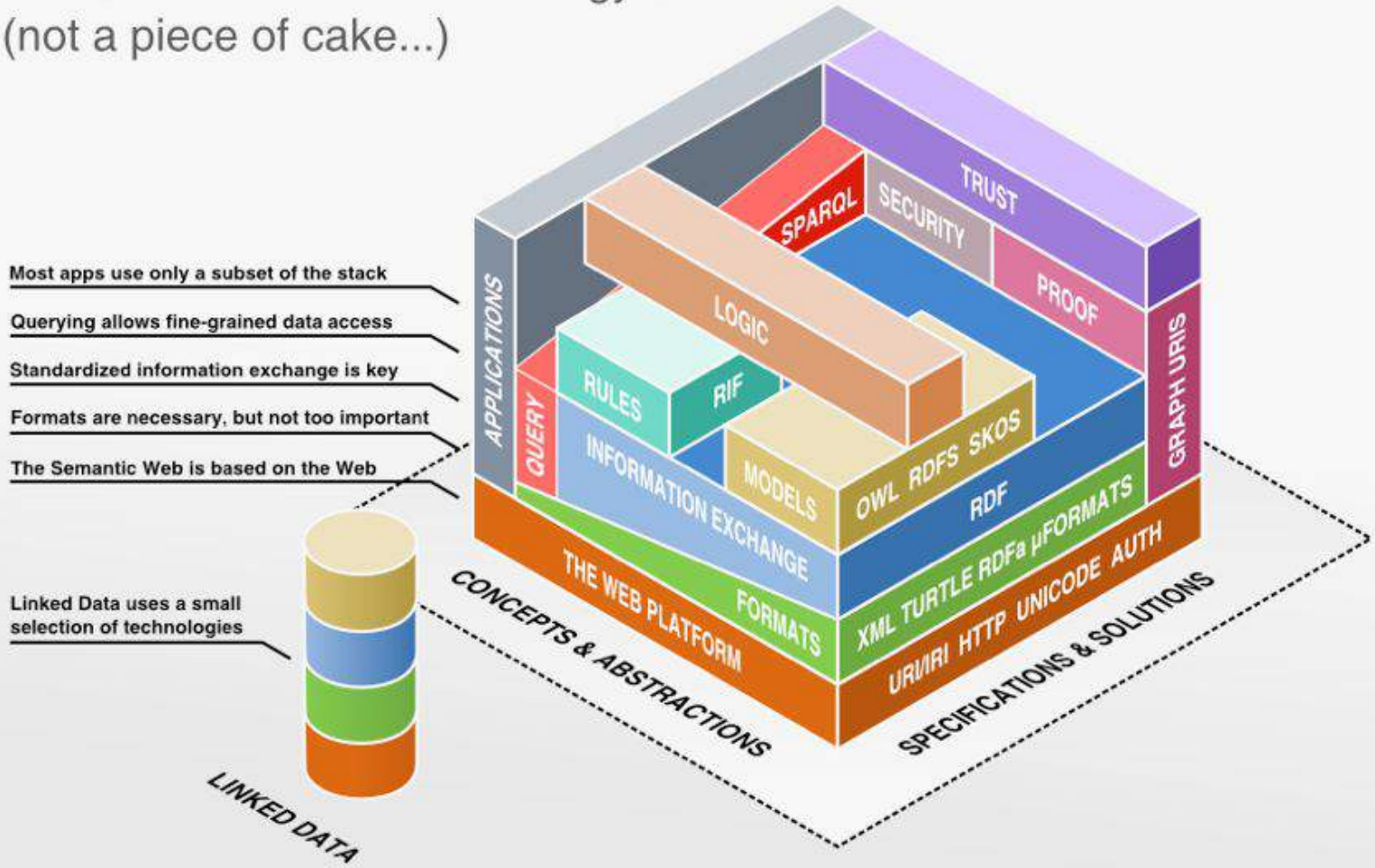
```
:satelliteOf   rdf:type    rdf:Property ;
               rdfs:domain :CelestialBody .
               rdfs:range  :CelestialBody .
```

Property Definitions

```
:Earth    rdf:type     :Planet .
:Moon     rdf:type     :Satellite ;
          :satelliteOf :Earth .
:Sputnik1 rdf:type     :ArtificialSatellite ;
          :satelliteOf :Earth ;
          rdfs:label   "Sputnik 1"@en ;
          rdfs:comment "the first artificial Earth satellite in 1957" .
```

Instance Definitions

The Semantic Web Technology Stack
(not a piece of cake...)

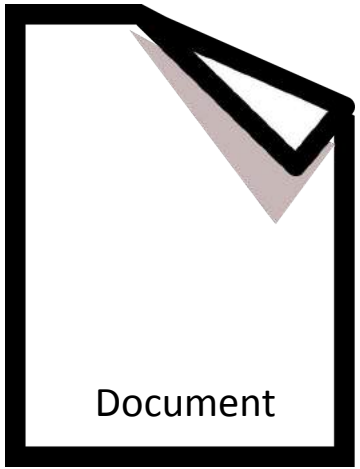


Vocabularies

- A person's **vocabulary** is the set of words within a language that are familiar to that person. (*Wikipedia*)
- = “all the words known and used by a particular person” (*Cambridge Advanced Learners Dictionary*)
- On the Semantic Web, **vocabularies** define the concepts and relationships used to describe and represent an area of concern. (*W3C*)
- **Vocabularies** are used to
  - **classify** the **terms** that can be used in a particular application,
  - **characterize** possible **relationships**, and
  - **define** possible **constraints** on using those terms.

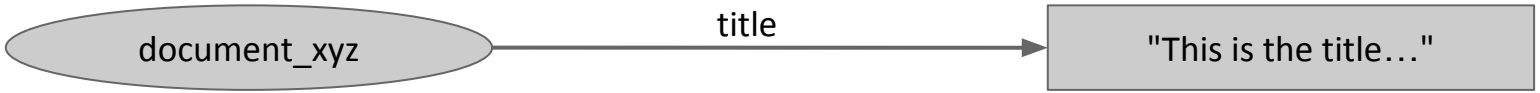
Example

- The **Dublin Core Vocabulary** (Dublin Core Metadata Element Set)



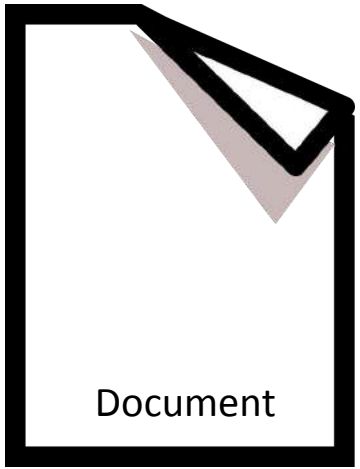
- Title
- Creator
- Subject
- Description
- Publisher
- Contributor
- Date

- Type
- Format
- Identifier
- Source
- Language
- Relation
- Coverage
- Rights



Example

- The **Dublin Core Vocabulary** (Dublin Core Metadata Element Set)



- Title
- Creator
- Subject
- Description
- Publisher
- Contributor
- Date
- Type
- Format
- Identifier
- Source
- Language
- Relation
- Coverage
- Rights

```
PREFIX dct: <http://purl.org/dc/terms/> .

<http://example.org/document_xyz> dct:title "This is the title…"@en .
```

Ontologies for Knowledge Representation

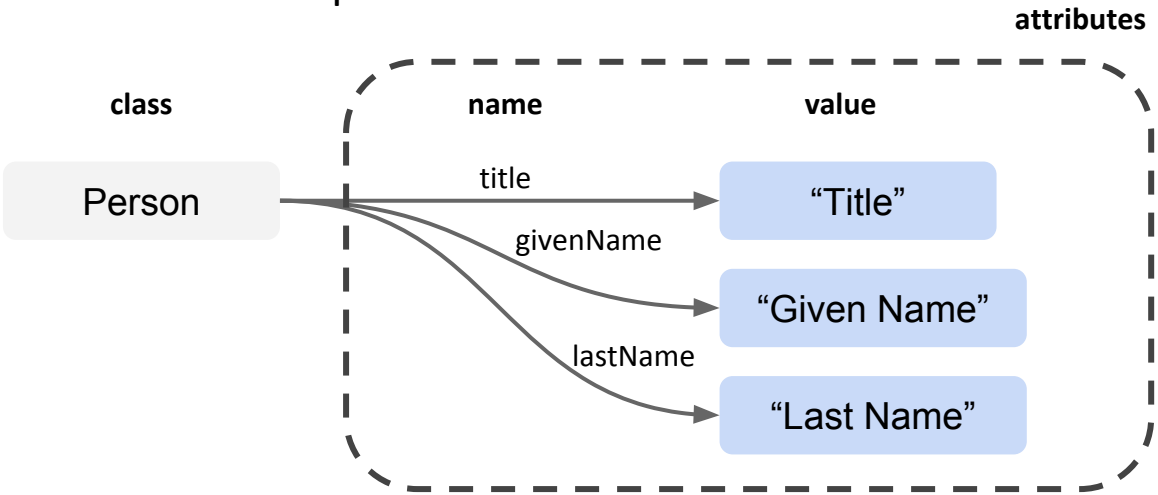
An ontology is an explicit, formal specification of a shared conceptualization. The term is borrowed from philosophy, where an Ontology is a systematic account of existence. For AI systems, what 'exists' is that which can be represented.

according to Thomas R. Gruber: A Translation Approach to Portable Ontology Specifications.Knowledge Acquisition, 5(2):199-220, 1993.

conceptualization:	abstract model (domain, identified relevant concepts, relations)
explicit:	meaning of all concepts must be defined
formal:	machine understandable
shared:	consensus about ontology

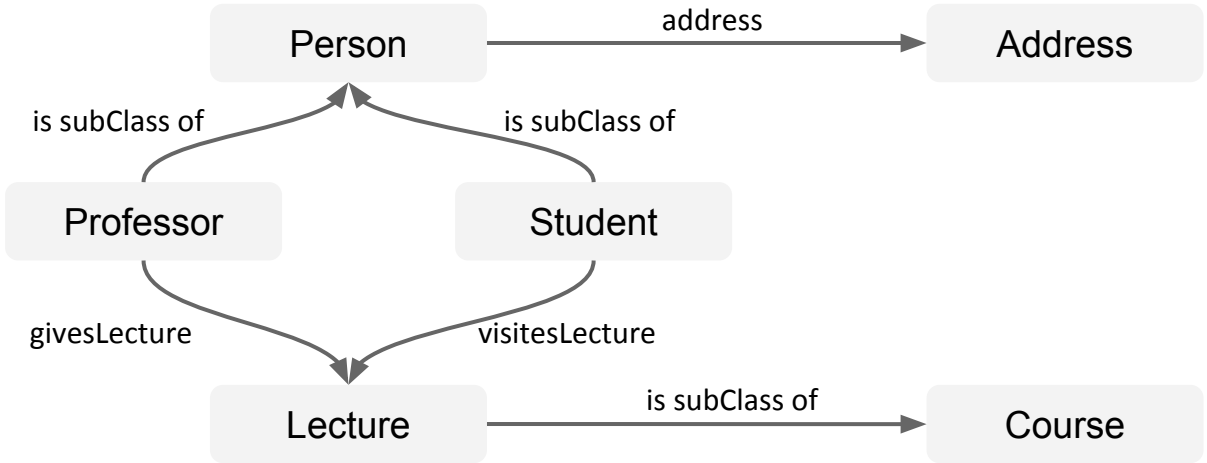
How to represent Ontologies

- Ontologies can be represented via **Classes, Relations** and **Instances**
- **Classes** are abstract groups, sets, or collections of objects and represent **ontology concepts**
- Classes are characterised via **attributes**
- **Attributes** are name-value pairs



How to represent Ontologies

- Classes can be **related** to other classes
- **Relations** are special attributes, whose values are objects of (other) classes



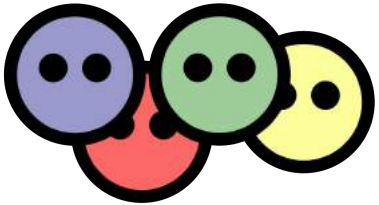
Example

- Defining personal data for a user/person
  - e.g. given name, last name, address data, web page, email address, etc.

- There are several possibilities:
  1. **Define a new vocabulary** according to your needs
  2. **Use existing vocabularies** and (possibly) adapt your application accordingly
  3. Use existing vocabularies and **extend vocabularies** according to your needs

Example of Linked Data Vocabularies

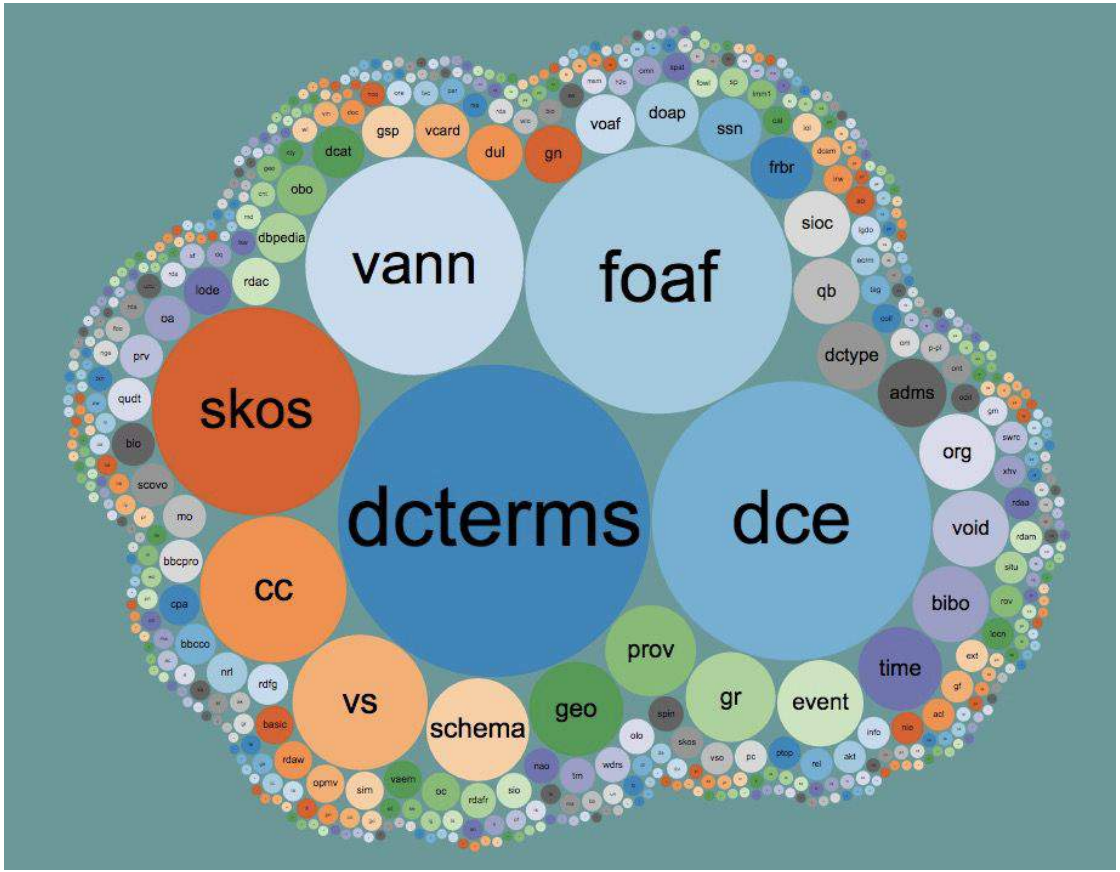
- **FOAF - Friend of a Friend**
  - describes persons, their activities and their relations to other people and objects



http://www.foaf-project.org/

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
<#me>
   a foaf:Person ;
   foaf:name "Harald Sack" ;
   foaf:mbox <mailto:harald.sack@hpi.de> ;
   foaf:homepage <https://hpi.de/meinel/lehrstuhl/team-fotos/senior-researcher/sack.html> ;
   foaf:depiction <https://hpi.de/fileadmin/_migrated/pics/harald_min.jpg> ;
   foaf:interest <http://linkeddata.org> ;
   foaf:knows [
      a foaf:Person ;
      foaf:name "Magnus Knuth"
   ] .
```

Where to find Linked Data Vocabularies



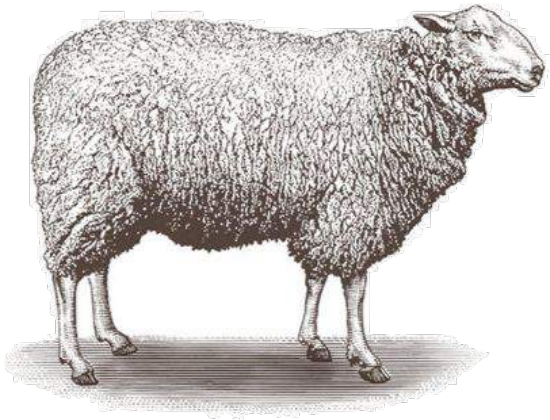
Linked Open Vocabularies
http://lov.okfn.org/dataset/lov/

The Open World Assumption - OWA

- Question: Can Sheep fly?
- Answer under OWA assumption:

```
A sheep is an animal with 4 legs.
```

No idea, but probably yes
(according to our knowledge base)

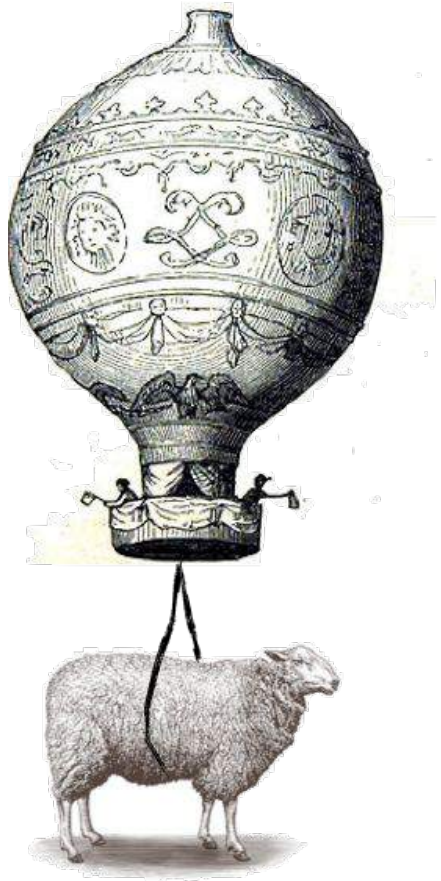


- In the OWA, unless we have a statement (or we can infer) "*sheep can/cannot fly*" we return "***don't know***"
- In the real world, we are used to deal with incomplete information

The Open World Assumption - OWA

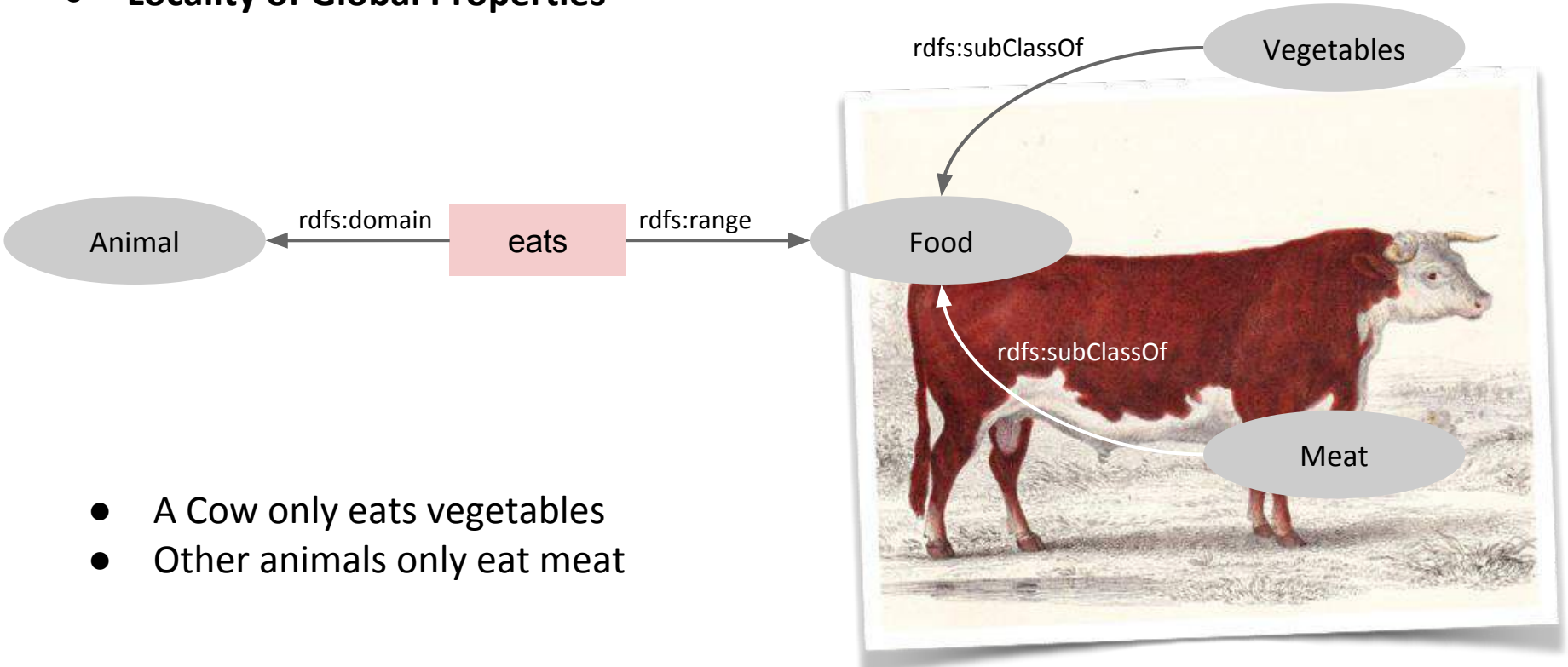
- In the Semantic Web / Linked Data we expect people to extend our own models
  (*but we don‘t worry in advance how*)
- The OWA assumes incomplete information by default
- Therefore, we can intentionally underspecify our models and allow others to reuse and extend

```
A sheep is an animal with 4 legs that
        (if lifted) can fly
```



Where RDF(S) is not sufficient...

- **Locality of Global Properties**



- A Cow only eats vegetables
- Other animals only eat meat

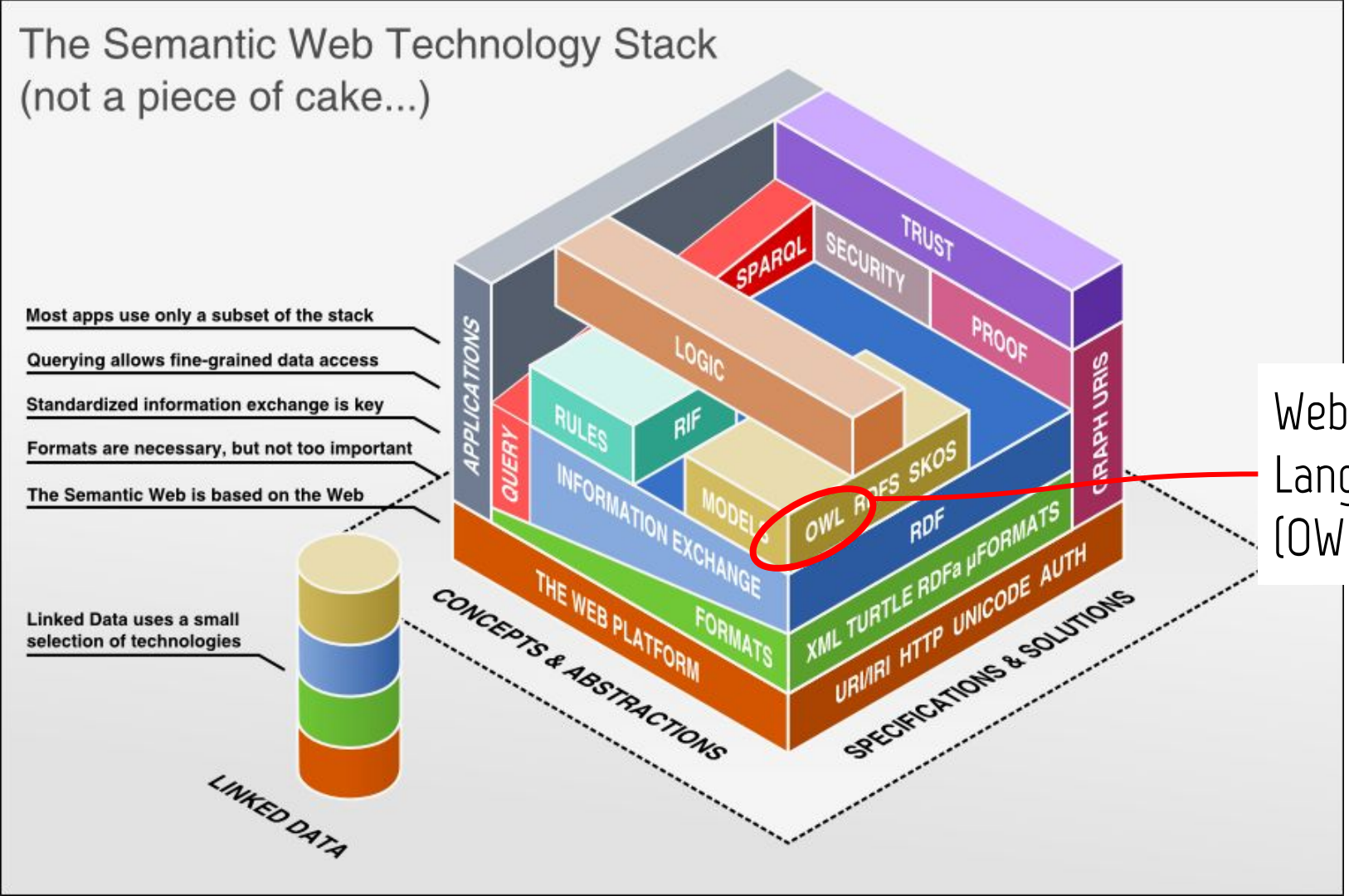
Where RDF(S) is not sufficient...

Special Property Constraints

- Transitivity (e.g. „is greater than“)
- Uniqueness (e.g. „is mother of“)
- Inversiveness (e.g. „is parent of“ and „is child of“)

General Problem of RDF(S)

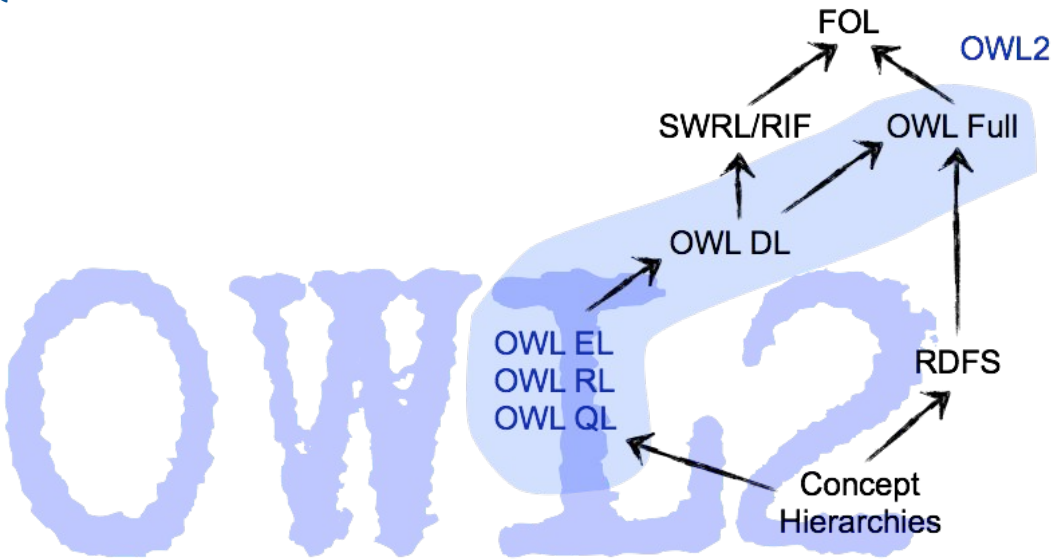
- RDF(S) does not have the possibility of **negation**
  - `hpi:harald rdf:type hpi:Vegetarian .`
  - `hpi:harald rdf:type hpi:NonVegetarian .`
  - ...does not automatically generate a contradiction



Web Ontology Language (OWL)

Web Ontology Language - OWL

- OWL is a semantic fragment of First Order Logic (FOL)
- OWL also exists in different flavors
  - OWL EL, OWL RL, OWL QL ⊆ OWL2 DL ⊆ OWL2 Full



OWL2 is based on the Description Logic $\mathcal{SROIQ}(\mathcal{D})$

Class Expressions

- Class names A, B
- Conjunction $C \sqcap D$
- Disjunction $C \sqcup D$
- Negation $\neg C$
- Exist. property restriction $\exists R.C$
- Univ. property restriction $\forall R.C$
- Self $\exists S.Self$
- Greater-than $\geq n\ S.C$
- Less-than $\leq n\ S.C$
- Enumerated classes $\{a\}$

Properties

- Property names R, S, T
- Simple properties S, T
- Inverse properties R^-
- Universal property U

Tbox (Class axioms)

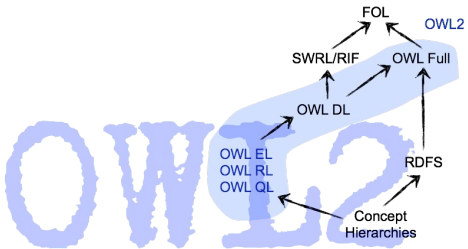
- Inclusion $C \sqsubseteq D$
- Equivalence $C \equiv D$

Rbox (Property Axioms)

- Inclusion $R_1 \sqsubseteq R_2$
- General Inclusion $R^{(-)}_1 \circ R^{(-)}_2 \circ \ldots \circ R^{(-)}_n \sqsubseteq R$
- Transitivity
- Symmetry
- Reflexivity
- Irreflexivity
- Disjunctiveness

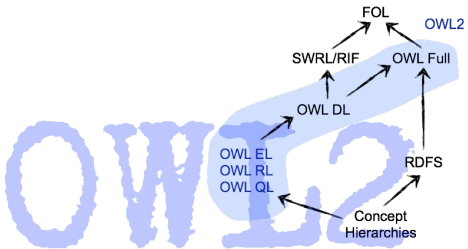
Abox (Facts)

- Class membership $C(a)$
- Property relation $R(a,b)$
- Negated property relation $\neg S(a,b)$
- Equality $a=b$
- Inequality $a \neq b$



OWL Basic Building Blocks

- OWL **namespace**:
  `@prefix owl: <http://www.w3.org/2002/07/owl#>`
- There is a **Turtle Syntax** for OWL
- OWL axioms consist of the following three building blocks:
  - **Classes**
    - comparable with classes in RDFS
  - **Individuals**
    - comparable with class instances in RDFS
  - **Properties**
    - comparable with properties in RDFS



OWL Classes

- there exist two predefined classes
  - `owl:Thing` (class that contains all individuals)
  - `owl:Nothing` (empty class)

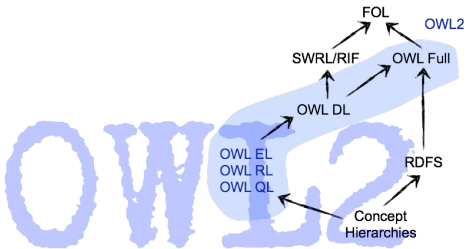
- Definition of a class

```
:Book a owl:Class .
```

equivalent expression in description logics

$\top \equiv C \sqcup \neg C$

$\bot \equiv C \sqcap \neg C$



OWL Individuals

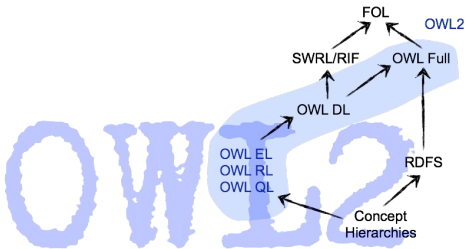
- Definition of individuals via class membership

```
:NineteenEightyfour a :Book .
```

```
Book(NineteenEightyfour)
```

- Individuals can also be defined without class membership as named individuals

```
:HaraldSack a owl:NamedIndividual .
```



OWL Object Properties

- there exist two property variants:
  - object properties
  - datatype properties

- **Object properties** have classes as range

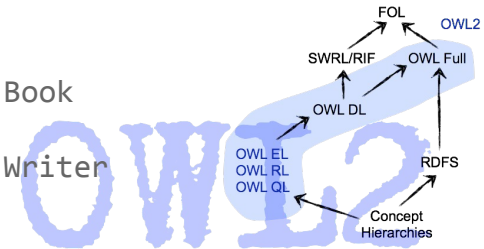
```
:author a owl:ObjectProperty .
```

- Domain and Range of object properties

```
:author a owl:ObjectProperty ;
        rdfs:domain :Book ;
        rdfs:range  :Writer .
```

$\exists \text{author}.\top \sqsubseteq \text{Book}$

$\top \sqsubseteq \forall \text{author}.\text{Writer}$



OWL Datatype Properties

- **Datatype properties** have datatypes as range

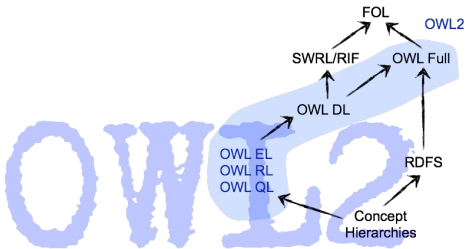
```
:publicationYear a owl:DatatypeProperty .
```

- Domain and Range of datatype properties

```
:publicationYear a owl:DatatypeProperty ;
                 rdfs:domain :Book ;
                 rdfs:range  xsd:integer .
```

$\exists \text{publicationYear}.\top \sqsubseteq \text{Book}$

$\top \sqsubseteq \forall \text{publicationYear}.\text{Integer}$



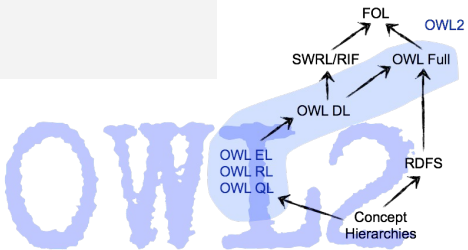
OWL Properties and Individuals

```
:Book a owl:Class .
:Writer a owl:Class .

:author a owl:ObjectProperty ;
        rdfs:domain :Book ;
        rdfs:range  :Writer .

:publicationYear a owl:DatatypeProperty ;
                 rdfs:domain :Book ;
                 rdfs:range xsd:integer .

:GeorgeOrwell a Writer .
:NineteenEightyFour a :Book ;
                    :author :GeorgeOrwell ;
                    :publicationYear 1948 .
```

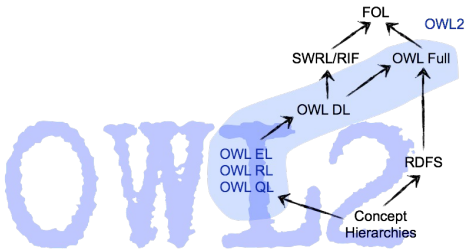


OWL Class Hierarchies

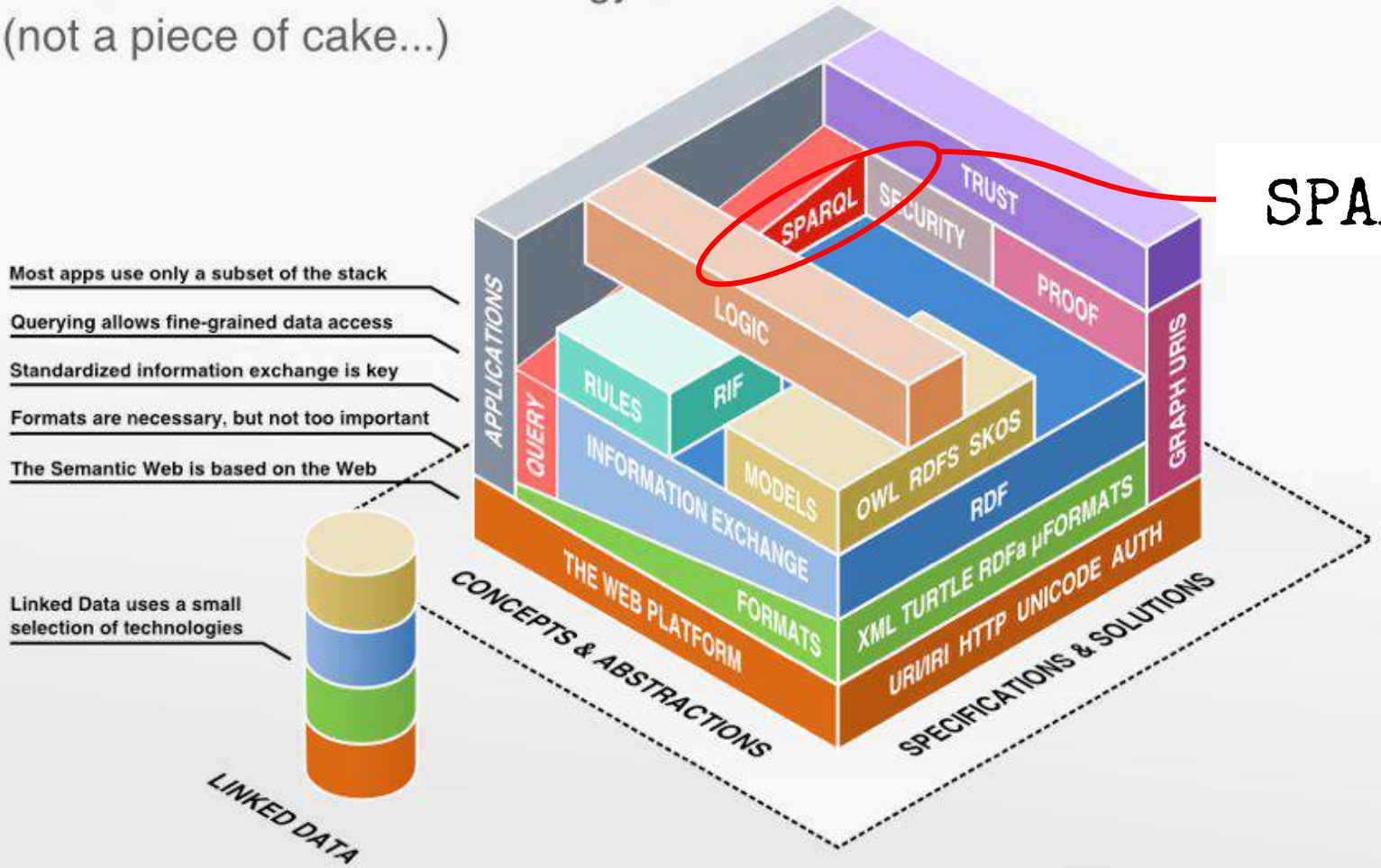
```
:Novel a owl:Class ;
       rdfs:subClassOf :Book .
:Book a owl:Class ;
      rdfs:subClassOf :Work .
:Work a owl:Class .
```

$$\texttt{Novel} \sqsubseteq \texttt{Book}$$
$$\texttt{Book} \sqsubseteq \texttt{Work}$$

- via inference it can be entailed that `:Novel` is also a subclass of `:Work`



The Semantic Web Technology Stack
(not a piece of cake...)



SPARQL

Most apps use only a subset of the stack

Querying allows fine-grained data access

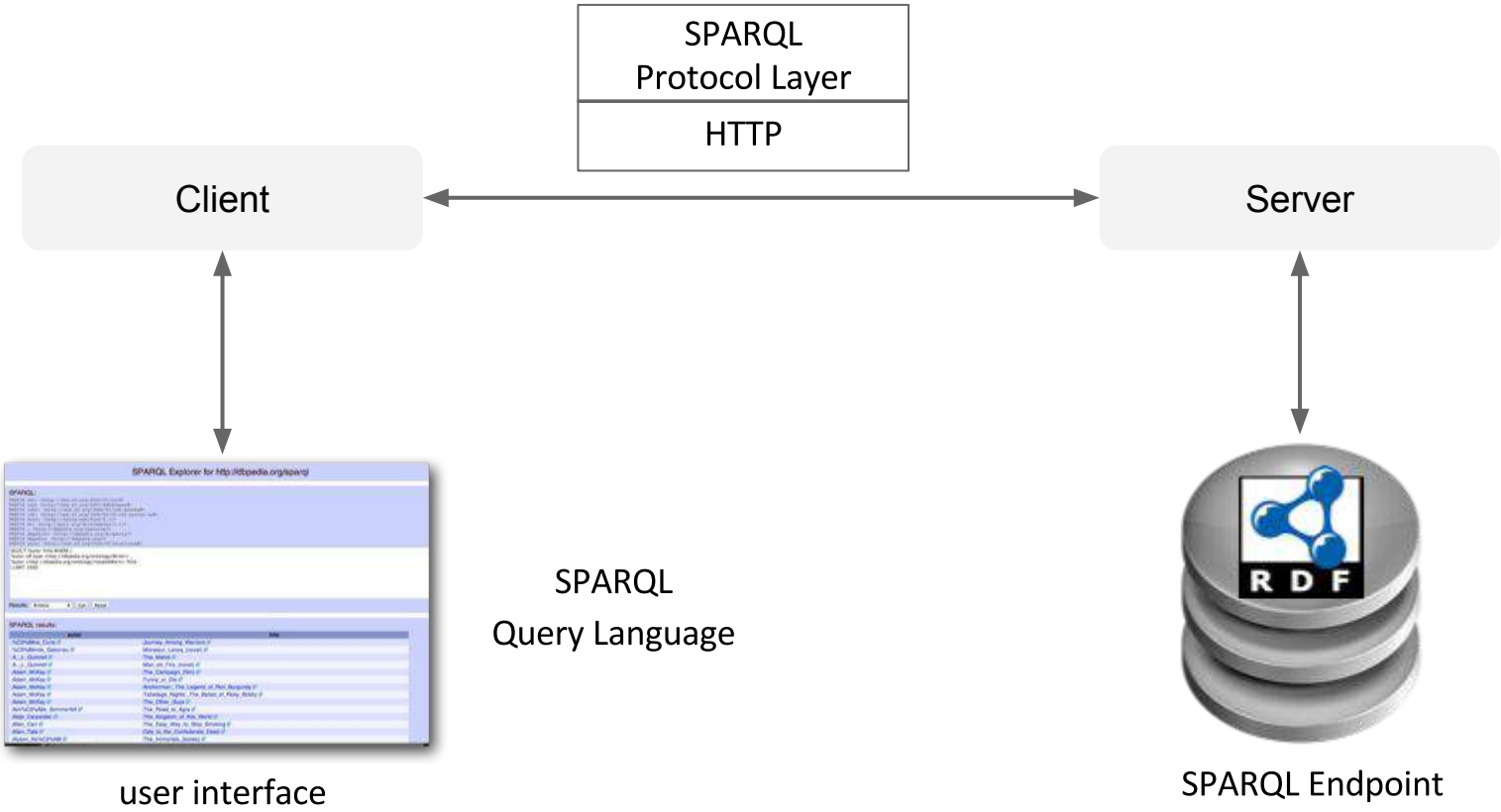
Standardized information exchange is key

Formats are necessary, but not too important

The Semantic Web is based on the Web

Linked Data uses a small selection of technologies

SPARQL - A Query Language for RDF



SPARQL - A Query Language for RDF

- **SPARQL P**rotocol **a**nd **R**DF **Q**uery **L**anguage is
  - a **Query Language** for RDF graph traversal (*SPARQL Query Language Specification*)
  - a **Protocol Layer**, to use SPARQL via http (*SPARQL Protocol for RDF Specification*)
  - an **XML Output Format Specification** for SPARQL queries (*SPARQL Query XML Results Format*)
  - W3C Standard (SPARQL 1.1, Mar 2013)
  - inspired by SQL

For Queries we need Variables

- SPARQL **Variables** are bound to RDF terms
  - e.g. **?title, ?author, ?address**
- In the same way as in SQL,
  a **Query for variables** is performed via **SELECT statement**
  - e.g. **SELECT ?title ?author ?published**

SPARQL Query

- A SELECT statement returns Query Results as a **table**

?title	?author	?published
1984	George Orwell	1948
Brave New World	Aldous Huxley	1932
Fahrenheid 451	Ray Bradbury	1953

SPARQL Result

SPARQL - Graph Pattern Matching

- SPARQL is based on **RDF Turtle serialization** and **basic graph pattern matching**.
- A **Graph Pattern** (**Triple Pattern**) is a RDF Triple that contains variables at any arbitrary place (Subject, Property, Object).

  (Graph) Triple Pattern = Turtle + Variables

- Example:
  Look for countries and their capitals:

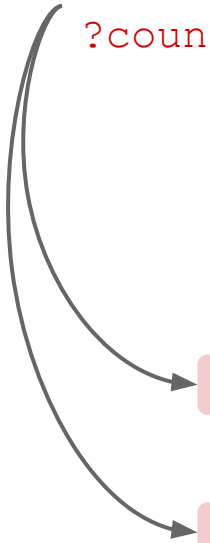
```
?country dbo:capital ?capital .
```

- A **Basic Graph Pattern** (**BGP**) is a set of Triple Pattern

SPARQL - Graph Pattern Matching

Triple Pattern

```
?country dbo:capital ?capital .
```



RDF Graph

```
dbpedia:Venezuela rdf:type dbo:Country .
dbpedia:Venezuela dbo:capital dbpedia:Caracas .
dbpedia:Venezuela dbprop:language "Spanish" .
dbpedia:Germany rdf:type dbo:Country .
dbpedia:Germany dbo:capital "Berlin" .
dbpedia:Germany dbp:language "German" .
...
```

SPARQL - Complex Query Patterns

- SPARQL Graph Pattern can be combined to form **complex (conjunctive) queries** for RDF graph traversal

- *Find countries, their capitals, and their population count:*

```
?country dbo:capital ?capital .
?country dbo:population ?population .
```

- *Given a FOAF URI, find the name of a person and her friends:*

```
<http://hpi-web.de/id#haraldsack> foaf:name  ?surname ;
                                  foaf:knows  ?friend .
?friend foaf:name ?friend_surname .
```

SPARQL - General Query Format

- *search all authors and the titles of their notable works:*

```
PREFIX :     <http://dbpedia.org/resource/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo:  <http://dbpedia.org/ontology/>

SELECT ?author_name ?title

FROM <http://dbpedia.org/>

WHERE {
     ?author rdf:type dbo:Writer .
     ?author rdfs:label ?author_name .
     ?author dbo:notableWork ?work .
     ?work rdfs:label ?title .
}
```

specifies namespaces

specifies output variables

specifies graph to be queried

specifies graph pattern to be matched

SPARQL - General Query Format

- *search all authors and the titles of their notable works **ordered by** authors in ascending order and **limit** the results to the first 100 results starting the list at **offset** 10 position:*

```
PREFIX :     <http://dbpedia.org/resource/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo:  <http://dbpedia.org/ontology/>

SELECT ?author_name ?title

FROM <http://dbpedia.org/>

WHERE {
      ?author rdf:type dbo:Writer .
      ?author rdfs:label ?author_name .
      ?author dbo:notableWork ?work .
      ?work rdfs:label ?title .
}
ORDER BY ASC (?author_name)
LIMIT 100
OFFSET 10
```

solution sequence modifiers

query SPARQL endpoint

SPARQL - Filter Constraints

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?author_name ?title ?pages
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       ?author rdfs:label ?author_name .
       ?author dbo:notableWork ?work .
       ?work dbo:numberOfPages ?pages .
       FILTER (?pages > 500)
       ?work rdfs:label ?title .
} LIMIT 100
```

specifies constraints for the result

- `FILTER` expressions contain operators and functions

query SPARQL endpoint

SPARQL - Unary Operator Constraints

Operator	Type(A)	Result Type
`!A`	xsd:boolean	xsd:boolean
`+A`	numeric	numeric
`-A`	numeric	numeric
`BOUND(A)`	variable	xsd:boolean
`isURI(A)`	RDF term	xsd:boolean
`isBLANK(A)`	RDF term	xsd:boolean
`isLITERAL(A)`	RDF Term	xsd:boolean
`STR(A)`	literal/URL	simple literal
`LANG(A)`	literal	simple literal
`DATATYPE(A)`	literal	URI

SPARQL - Filter Constraints

- Example: Filter results only for English labels

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?author_name ?title ?pages
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       ?author rdfs:label ?author_name
       FILTER (LANG(?author_name)="en").
       ?author dbo:notableWork ?work .
       ?work dbo:numberOfPages ?pages .
       FILTER (?pages > 500)
       ?work rdfs:label ?title .
       FILTER (LANG(?title)="en")
} LIMIT 100
```

query SPARQL endpoint

SPARQL - Filter Constraints

- Example: Filter results only for English labels

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?author_name ?title ?pages
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       ?author rdfs:label ?author_name .
       FILTER (LANG(?author_name)="en")
       ?author dbo:notableWork ?work .
       ?work dbo:numberOfPages ?pages .
       FILTER (?pages > 500)
       ?work rdfs:label ?title .
       FILTER (LANG(?title)="en")
} LIMIT 100
```

More SPARQL Operators

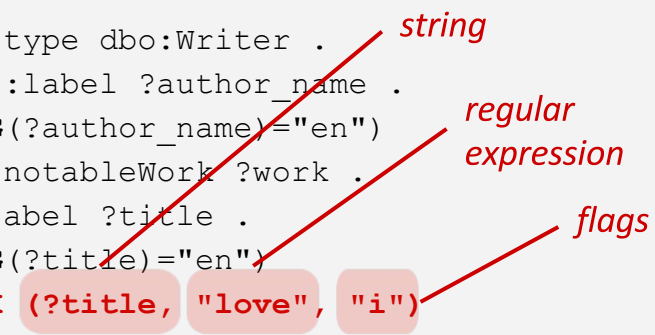
- Logical connectives `&&` and `||` for xsd:boolean
- Comparison operators `=`, `!=`, `<`, `>`, `<=`, and `>=` for numeric datatypes, xsd:dateTime, xsd:string, and xsd:boolean
- Comparison operators `=` and `!=` for other datatypes
- Arithmetic operators `+`, `-`, `*`, and `/` for numeric datatypes

- and in addition:
  - `REGEX(String,Pattern)` or `REGEX(String,Pattern,Flags)`
  - `sameTERM(A,B)`
  - `langMATCHES(A,B)`

SPARQL - Filter Constraints

- Example: Book titles that contain the word “love”

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?author_name ?title
FROM <http://dbpedia.org/>
WHERE {
        ?author rdf:type dbo:Writer .
        ?author rdfs:label ?author_name .
        FILTER (LANG(?author_name)="en")
        ?author dbo:notableWork ?work .
        ?work rdfs:label ?title .
        FILTER (LANG(?title)="en")
        FILTER REGEX (?title, "love", "i")
} LIMIT 100
```



learn more about regular expressions at http://regexone.com/

query SPARQL endpoint

SPARQL - Filter Constraints

- Example: Retrieve also the German book title, **if available**

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?author_name ?en_title ?de_title
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       ?author rdfs:label ?author_name .
       FILTER (LANG(?author_name)="en")
       ?author dbo:notableWork ?work .
       ?work rdfs:label ?en_title .
       FILTER (LANG(?en_title)="en")
       OPTIONAL { ?work rdfs:label ?de_title .
                  FILTER (LANG(?de_title)="de")
       }
} LIMIT 100
```



- The keyword **OPTIONAL** selects optional elements from the RDF graph
- complies to a Left Outer Join

query SPARQL endpoint

SPARQL - Negation

- Example: Retrieve authors that don‘t have an entry for “notable work”

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>

SELECT ?author
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       OPTIONAL { ?author dbo:notableWork ?work . }
       FILTER (!BOUND(?work))
} LIMIT 100
```

no variable binding

- Negation in SPARQL
- complies to “NOT EXISTS” in SQL

query SPARQL endpoint

SPARQL - Negation (2)

- Example: Retrieve authors that don‘t have an entry for “notable work”

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>

SELECT ?author
FROM <http://dbpedia.org/>
WHERE {
       ?author rdf:type dbo:Writer .
       FILTER NOT EXISTS {?author dbo:notableWork ?work .}

} LIMIT 100
```

filter query result for existency

- SPARQL 1.1 also provides `FILTER` expressions `NOT EXISTS` and `EXISTS`

SPARQL - Negation (3)

- Example: Retrieve authors that don‘t have an entry for “notable work”

```
PREFIX : <http://dbpedia.org/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX dbo: <http://dbpedia.org/ontology/>

SELECT ?author
FROM <http://dbpedia.org/>
WHERE {
        ?author rdf:type dbo:Writer .
        MINUS {?author dbo:notableWork ?work .}

} LIMIT 100
```

remove from query result

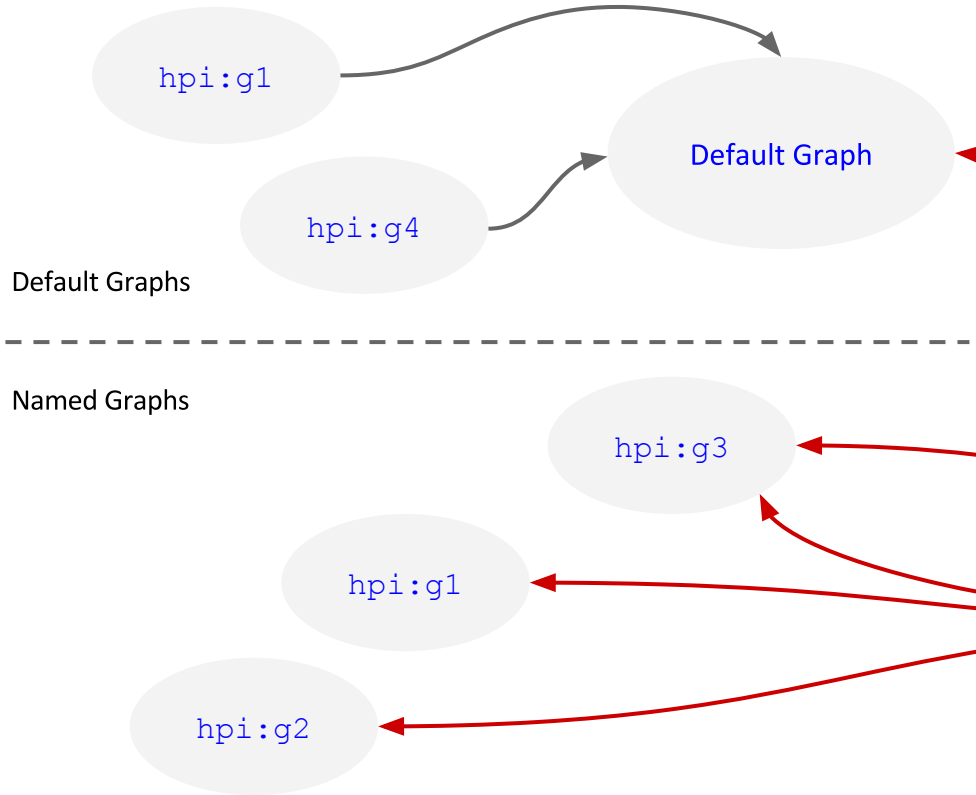
- Filtering of query solutions by removing possible solutions with `MINUS`.
- Difference to `NOT EXIST`:
- `MINUS` changes the graph pattern
- query result is dependent on position of `MINUS`

SPARQL - RDF Graphs

- SPARQL queries are executed over an RDF dataset
  - one (or more) **default RDF graph** (`FROM`)
  - zero or more **named RDF graphs** (`FROM NAMED`, `GRAPH`)
- **Named Graphs** can be explicitly addressed via the keyword `GRAPH` and the URI of the named graph

```
SELECT ...
WHERE {
  ...
  GRAPH <http://example.org/graph1.rdf> {
    ?x foaf:mbox ?mbox .
  }
  ...
}
```

SPARQL - RDF Graphs




```
PREFIX hpi: ...
SELECT ...
FROM hpi:g1
FROM hpi:g4
FROM NAMED hpi:g1
FROM NAMED hpi:g2
FROM NAMED hpi:g3
WHERE {
  ... A ...
  GRAPH hpi:g3 {
    ... B ...
  }
  GRAPH ?g {
    ... C ...
  }
}
```

SPARQL - RDF Graphs

- How to ask a SPARQL Endpoint which RDF Graphs are available?

```
SELECT DISTINCT ?g
WHERE {
  GRAPH ?g { ?s ?p ?o . }
}
```

query SPARQL endpoint

SPARQL - Federated Queries

- SPARQL enables federated queries over several RDF datasets or SPARQL endpoints via the **SERVICE** objective

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX movie: <http://data.linkedmdb.org/resource/movie/>
PREFIX dcterms: <http://purl.org/dc/terms/>

SELECT ?film ?label ?subject WHERE {
    SERVICE <http://data.linkedmdb.org/sparql> {
        ?film a movie:film .
        ?film rdfs:label ?label .
        ?film owl:sameAs ?dbpediaLink .
        FILTER regex(STR(?dbpediaLink), "dbpedia", "i")
    }
    SERVICE <http://dbpedia.org/sparql> {
        ?dbpediaLink dcterms:subject ?subject .
    }
}
LIMIT 100
```

- Example: connect the Linked Movie Database with DBpedia
- only possible, if SPARQL endpoints permit federation

RDF in RDBMS

- RDF Graph can be represented by a set of Triples (s,p,o)
- Triples can simply be stored in a **relational database management system** (RDBMS)
- **Motivation**: Use a specific relational schema for RDF data and benefit from 40 years of research in the DB community
- **3 steps for SPARQL query processing**:

  (1) Convert SPARQL query to SQL query (w.r.t. the schema)

  (2) Use RDBMS to answer SQL query

  (3) Generate SPARQL query result from SQL query result

RDF in RDBMS - The Problem

- How to store RDF triples to carry out efficient SPARQL queries?

- 4 alternatives:
  (1) Monolithic Triple Storage
  (2) Property Tables
  (3) Vertically Partitioned Tables (Binary Tables)
  (4) Hexastore

Monolithic Triple Storage

Subj	Property	Obj
ID1	type	FullProfessor
ID1	teacherOf	'AI'
ID1	bachelorFrom	'MIT'
ID1	mastersFrom	'Cambridge'
ID1	phdFrom	'Yale'
ID2	type	AssocProfessor
ID2	worksFor	'MIT'
ID2	teacherOf	'DataBases'
ID2	bachelorsFrom	'Yale'
ID2	phdFrom	'Stanford'
ID3	type	GradStudent
ID3	advisor	ID2
ID3	teachingAssist	'AI'
ID3	bachelorsFrom	'Stanford'
ID3	mastersFrom	'Princeton'
ID4	type	GradStudent
ID4	advisor	ID1
ID4	takesCourse	'DataBases'
ID4	bachelorsFrom	'Columbia'

- Basic Idea
  - Store all RDF Triples in a single table
  - performance depends on efficient indexing
- Pros:
  - easy to implement
  - works for huge number of properties, if indexes are chosen with care
- Cons:
  - many self joins necessary

Monolithic Triple Storage

Subj	Property	Obj
ID1	type	FullProfessor
ID1	teacherOf	'AI'
ID1	bachelorFrom	'MIT'
ID1	mastersFrom	'Cambridge'
ID1	phdFrom	'Yale'
ID2	type	AssocProfessor
ID2	worksFor	'MIT'
ID2	teacherOf	'DataBases'
ID2	bachelorsFrom	'Yale'
ID2	phdFrom	'Stanford'
ID3	type	GradStudent
ID3	advisor	ID2
ID3	teachingAssist	'AI'
ID3	bachelorsFrom	'Stanford'
ID3	mastersFrom	'Princeton'
ID4	type	GradStudent
ID4	advisor	ID1
ID4	takesCourse	'DataBases'
ID4	bachelorsFrom	'Columbia'

- Basic Idea
  - Store all RDF Triples in a single table
  - performance depends on efficient indexing

SPARQL

```
SELECT ?university WHERE {
   ?v rdf:type :GradStudent;
     :bachelorsFrom ?university . }
```

SQL

```
SELECT t2.o AS university
FROM triples AS t1, triples AS t2
WHERE t1.p='type' AND
      t1.o='GradStudent' AND
      t2.p='bachelorsFrom' AND
      t1.s=t2.s
```

ID Based Triple Storage

RDF Term	ID
:ID1	1
rdf:type	2
:FullProfessor	3
:teacherOf	4
“AI”	5
:bachelorsFrom	6
“MIT”	7
:mastersFrom	8
“Cambridge”	9

S	P	O
1	2	3
1	4	5
1	6	7
1	8	9
1	10	11
12	13	14
12	15	7
12	4	16

- **Resource table** for indexing URIs and Literals with numerical identifier
- **RDF Triple table** uses numerical identifier for each RDF term in the dataset
- saves space and enhances efficiency